Intelligent Systems (2802ICT) Assignment 2

Roman Green (s5369389)

Contents

Task 1 – Learning Decision Trees 2
1. Software Design 2
2. Experiments 5
3. Conclusion 7
Task 2 – Classification using a Neural Network 7
1. Software Design 7
2. Experiments 11
3. Conclusion 15
Appendix 1 Task 1 Source Code 16
1. decision_tree.py 16
2. helpers.py 17
Appendix 2 Task 2 Source Code 20
1. nn.py 20
2. nn_accelerated.py 22
Appendix 3 Task 1 eval.ipynb 26
Appendix 4 Task 2 planning.ipynb 28
Appendix 5 Task 2 running_nn.ipynb 33
Appendix 6 Task 2 benchmarking-nn.ipynb 37
Appendix 7 Task 2 benchmarking-nn-accelerated.ipynb 38
Appendix 8 Task 2 hyperparameter-tuning.ipynb 39

Task 1 – Learning Decision Trees

1. Software Design

1.1. Overview

The Iterative Dichotomiser 3 (ID3) algorithm was implemented to learn a decision tree from a sample dataset of car valuations.

The python data loading library polars (modern pandas equivalent) was used for csv loading, inspection and manipulation.

Two python files were created:

`decision_tree.py`	Implements the core methods for creating the decision tree.
`helpers.py`	Implements helper methods for metric computation.

Source code for both python files may be found in Appendix 1 Task 1 Source Code.

In addition, the following Jupyter notebook was created:

`eval.ipynb`	Jupyter notebook to complete experimentation requirements. Contains python blocks for loading the dataset, randomising the dataset into training and testing subsets and graphing. A copy of the Jupyter notebook is available in Appendix 3 Task 1 eval.ipynb.

1.2. Data Structures

The following data structures were defined:

`Status`	Type variable that defines the outcome class values, used for ensuring type safety: `Literal["unacc", "acc", "good", "vgood"]`
`InteriorNode`	Dataclass variable used as an interior node when building the decision tree. Contains three member variables: • attribute: the attribute column of the data for decision making. • plurality_prediction: the outcome class value to use when the example attribute value does not exist in the node's internal decision tree. • decision_tree: a dictionary mapping values for the given attribute to other nodes.
`LeafNode`	Dataclass variable used as a leaf node when building the decision tree. Contains one member variable: • value: the outcome class value to use when this node is reached.
`Node`	A union type containing `LeafNode` and `InteriorNode`.
`Metric`	Dataclass variable used to keep track of true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN) when computing accuracy, precision, recall and f1 score. A `Metric` dataclass will be instantiated for each outcome class value. Contains four member variables: • TP

- FP
- TN
- FN

`Metrics`	Class used to manage multiple Metric variables. Contains methods to compute macro and weighted precision, recall and f1 scores.

1.3. Key Functions

1.3.1. decision_tree.py

The following functions were implemented:

- `entropy(examples: pl.DataFrame) -> float`

Calculates the entropy of a given polars dataframe. First the probabilities of each outcome class are calculated, then total entropy is calculated as follows:

$$H(V) = -\sum_{k} P(v_k)\, log_2\big(P(v_k)\big)$$

Where V represents car valuation and $P(v_k)$ represents the probability of valuation k.

- `remainder(examples: pl.DataFrame, attribute: str) -> float`

Calculates the expected entropy in the dataset when partitioned by a given attribute. The dataframe is split into subsets for each unique value of the specified attribute, then the entropy of each subset dataframe is computed by calling the `entropy` function. Finally, average entropy weighted by height of each subset dataframe.

- `importance(examples: pl.DataFrame, attribute: str)`

Calculates the reduction in entropy in the dataset when partitioning by a given attribute by computing the difference `entropy(examples)` and `remainder(examples, attribute)`.

- `plurality_value(examples: pl.DataFrame) -> Status`

Determines the most common outcome class for the given dataframe.

- `ID3(examples: pl.DataFrame, attributes: set[str], parent_examples: pl.DataFrame) -> Node`

A recursive function that returns a `Node` for construction of the decision tree. If the given examples have know row, returns a LeafNode with value equal to the plurality value of parent_examples. If the given examples only have one outcome class value, returns a LeafNode with value equal to the one outcome class value. If the attribute set is empty, returns a LeafNode with value equal to the plurality value of the examples.

Otherwise, the ID3 will select the attribute with the highest value given by importance and construct an InteriorNode. The plurality_value of the interior node is set to the plurality_value of the given examples. Then for every value of the attribute, a new subtree is computed by calling ID3 and the internal decision tree of the InteriorNode is updated with each value:subtree pair. The InteriorNode is then returned.

- `classify(row: dict[str, Any], node: Node) -> Status`

Recursive function to traverse the decision tree and return the predicted outcome class value.

- `pretty_print_tree(node: Node, indent: str = "")`

Prints a decision tree to the console for debugging.

- `main()`

Entry point to the program.

While the program supports command line invocation, experiments were done through a Jupyter notebook.

1.3.2. helpers.py

The following functions were implemented:

- `Metric.total(self) -> int`

Returns the number of examples processed by this class: $TP + FP + TN + FN$.

- `Metric.accuracy(self) -> float`

Returns the accuracy of the metric: $(\mathrm{TP} + \mathrm{TN})/\mathrm{total}$.

- `Metric.precision(self) -> float`

Returns the precision of the metric: $(TP + FP)/total$.

- `Metric.recall(self) -> float`

Returns the recall of the metric: $TP/(TP + FN)$.

- `Metric.f1_score(self) -> float`

Returns the f1 score of the metric: $\frac{2 \cdot \mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}$.

- `Metrics.update(self, model_prediction: str, ground_truth: str)`

Takes a model prediction and the ground truth label, and iterates over each metric for each outcome class to update their internal TP, TN, FP, FN counts.

- `Metrics.print_summary(self)`

Computes metrics and returns as an ascii table:

```
[23]  m.print_summary()
      ✓ 0.0s
...
                    precision    recall     f1-score    support
-----------------------------------------------------------------------------
unacc               0.9882       0.9582     0.9730      263
acc                 0.8228       0.9286     0.8725      70
good                0.8182       0.6000     0.6923      15
vgood               0.7273       1.0000     0.8421      8
-----------------------------------------------------------------------------
macro avg           0.8391       0.8717     0.8450      356
weighted avg        0.9427       0.9382     0.9384      356
-----------------------------------------------------------------------------

accuracy            0.9382
```

- `Metrics.total(self) -> int`

Returns the number of examples processed by this class. Equivalent to `Metric.total()`.

- `Metrics.accuracy(self) -> float`

Returns the accuracy of classifications processed by the class. Internally it sums the TP of each metric and divides by total.

- `Metrics.average_results(self) -> dict[str, float]`

Returns macro and weighted precision/recall/f1 score. Macro precision/recall/f1 score simply computes the average precision/recall/f1 score of the stored `Metric`'s. Weighted precision/recall/f1 score computes the average precision/recall/f1 score of the stored `Metric`'s weighted by the support of each `Metric`.

1.4. Optimisations

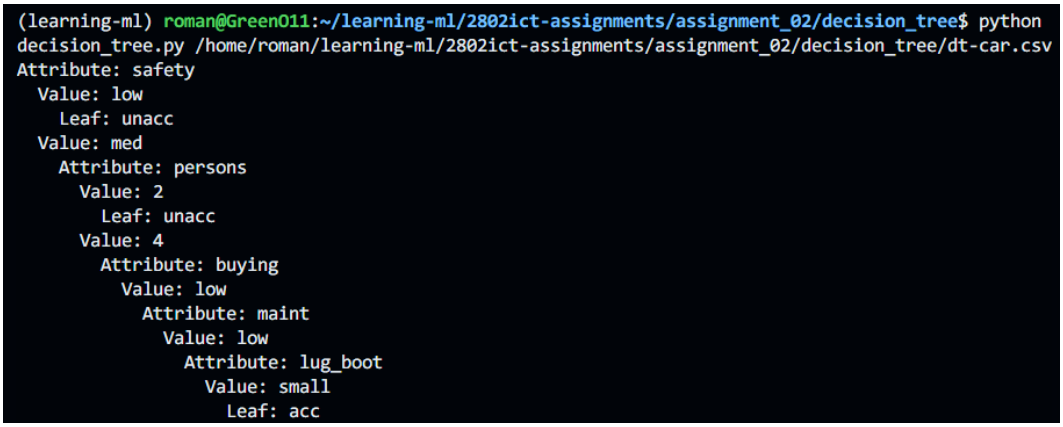
The implementation was optimised primarily by leveraging the polars library. The Rust-based engine executes vectorized operations on data columns, which is faster than iterative Python approaches for the frequent data filtering and grouping.

1.5. Screenshots Showing Program Execution

While the model implementation supports execution via command line invocation, Jupyter notebooks were used for conducting experiments and graphing. Jupyter notebooks may be found in the following appendices:

- Appendix 3 Task 1 eval.ipynb

Figure *1* shows command line invocation.



```
(learning-ml) roman@Green011:~/learning-ml/2802ict-assignments/assignment_02/decision_tree$ python
decision_tree.py /home/roman/learning-ml/2802ict-assignments/assignment_02/decision_tree/dt-car.csv
Attribute: safety
  Value: low
    Leaf: unacc
  Value: med
    Attribute: persons
      Value: 2
        Leaf: unacc
      Value: 4
        Attribute: buying
          Value: low
            Attribute: maint
              Value: low
                Attribute: lug_boot
                  Value: small
                    Leaf: acc
```

Figure 1 Screenshot showing `decision_tree.py` command line invocation.

2. Experiments

2.1. Base Case

The dataset (n=1728) was randomly partitioned into train (n=1380) and test (n=348) subsets with a partitioning ration of 0.8.

A decision tree was then computed for the train dataset.

Table 1 shows the decision tree performance evaluation on the test dataset. The model demonstrated a high overall accuracy of 92.24%. However, due to a class imbalance in the test dataset, model performance is skewed by its ability to predict the majority class unacc (F1 score=0.9631). The model shows poor performance on minority classes, the F1 score for acc, good and vgood is 0.8477, 0.7586 and 0.7857 respectively. The macro average (0.8060) provides a more practical assessment of model performance by weighting all classes equally.

Table 1 Learned Decision Tree Performance Evaluation

	Precision	Recall	F1-score	Support
unacc	0.9671	0.9592	0.9631	245
acc	0.8767	0.8205	0.8477	78
good	0.6471	0.9167	0.7586	12
vgood	0.7333	0.8462	0.7857	13
macro avg	0.8060	0.8856	0.8388	348
weighted avg	0.9271	0.9224	0.9236	348

2.2. Accuracy vs Training Size

The model was trained repeatedly on increasing fractions of the training data and evaluated on a hold-out test set (n=348). Figure 2 Shows the impact of training dataset size on validation accuracy. There is a consistent positive correlation between the training dataset size and the model's accuracy. The curve trends upwards from approximately 70% with very few examples to over 90% as the full dataset is utilized. The steady increase in performance with no plateau (shown on Figure 2 Graph A) suggests the model is successfully generalising and may benefit from a larger training set size.

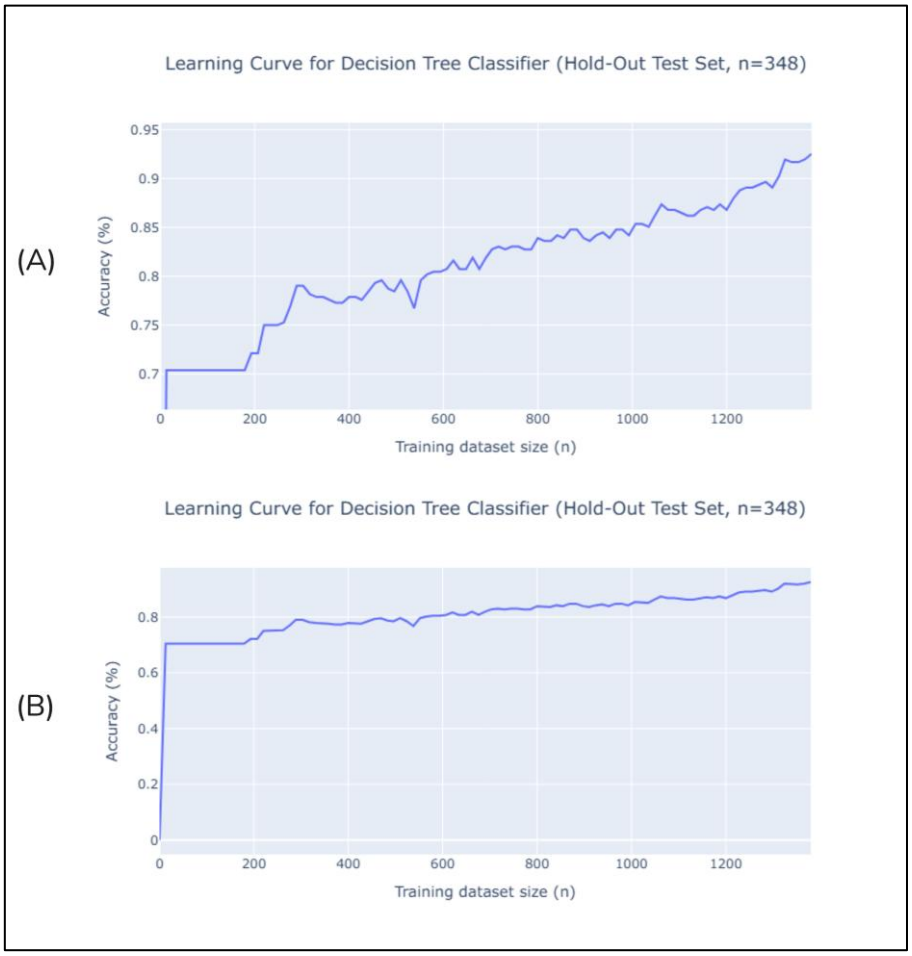


Figure 2 Impact of training dataset size on validation accuracy. A decision tree was repeatedly learnt for increasing fractions (x) of the training data (n=1380; x=0.00, 0.01, …, 1.00) and evaluated on a hold-out test set (n=348). Graph A shows performance over a constrained y-axis (0.6 – 0.96). Graph B shows performance over an unconstrained y-axis (0.0 – 1.0).

3. Conclusion

The Iterative Dichotomiser 3 (ID3) algorithm was successfully implemented in Python to build a decision tree classifier for the car valuation dataset. Evaluation of the model on a hold-out test set achieved a high overall accuracy of 92.24%. However, analysis of per-class metrics revealed that this performance was skewed by class imbalance, with the model performing significantly better on the majority class ('unacc') than on minority classes. The learning curve experiment demonstrated a clear positive correlation between training data size and model accuracy, indicating that the classifier generalises well and could benefit from additional training data.

Task 2 – Classification using a Neural Network

1. Software Design

1.1. Overview

A fully connected 2-layer (input layer → hidden layer → output layer) neural network was implemented for arbitrary classification tasks. The implemented model supports adjustment of parameters such as input neuron count, hidden neuron count, output neuron count, number of training epochs, mini-batch size and learning rate. Each trainable layer utilises the sigmoid activation function and no weight decay or dropout was used.

Softmax is applied to model outputs to determine prediction probabilities of each class:

$$\hat{y}_{\mathrm{i}} = \frac{e^{o_i}}{\sum_k e^{o_k}}$$

Where $\hat{y}_{\mathrm{i}}$ is the predicted probability for class i, o_i is the logit (raw output) of the model for class i and o_k is the logit for class k used in the denominator to sum over all classes.

The model is trained to minimise cross entropy loss:

$$l = -\sum_{\mathrm{i}=1}^{q} y_{\mathrm{i}} \cdot \log(\hat{y}_{\mathrm{i}})$$

Where l is the loss for a single example, q is the number of classes and y_{i} is the true label.

A more complete description of the operations underpinning the model can be found in Appendix 4 Task 2 planning.ipynb.

Two python files were created:

`nn.py`	Implements the forward and backward passes for training the neural network.
`nn_accelerated.py`	Reimplementation of `nn.py` in JAX for GPU acceleration.

Source code for both python files may be found in Appendix 2 Task 2 Source Code.

In addition, the following Jupyter notebooks were created:

`planning.ipynb`	Jupyter notebook for planning out the forward and backward passes in vectorised, batch-level form.
`running-nn.ipynb`	Jupyter notebook to complete experimentation requirements. Contains python code blocks and

	python graphing utilities for (experiment 1) training the model, (experiment 2) training the model while varying learning rate, and (experiment 3) training the model while varying mini-batch size.
`benchmarking-nn.ipynb`	Jupyter notebook to measure `nn.py` training time.
`benchmarking-nn-accelerated.ipynb`	Jupyter notebook to measure `nn_accelerated.py` training time.
`hyperparameter-tuning.ipynb`	Jupyter notebook for evaluating optimal model parameters.

1.2. Data Structures

Please note that the implementation of these data structures may differ between the `nn.py` and `nn_accelerated.py` files. The implementation in `nn_accelerated.py` was modified to be purely functional as is required by JAX tracing for just-in-time compilation. For the purposes of this document, any descriptions of any data structures or key functions refer to the `nn.py` implementation.

`Model`	Dataclass variable used to keep model weights and biases. Contains four member variables: • W_1: numpy array holding weights for layer 1. • B_1: numpy array holding biases for layer 1. • W_2: numpy array holding weights for layer 2. • B_2: numpy array holding biases for layer 2.
`RunningAverage`	Utility dataclass to for moving average tracking. Contains two methods: • update(self, new_value): updates the stored running average with the incoming value. • reset(self): resets the stored running average.
`ModelMetrics`	Class used to track loss and accuracy of the model. Internally it instantiates two variables with types: • loss: `RunningAverage` • accuracy: `RunningAverage` Provides two methods: • update(self, y, y_hat): updates internal loss and accuracy values using the ground truth labels (y) and model predictions (y_hat). • reset(self): resets loss and accuracy to default values.

1.3. Data Representations

Labels are one hot encoded to simplify loss computation. Training data is passed to the train function in batches. Each batch conforms to the following shape: `tuple[np.ndarray, np.ndarray]`. The first `np.ndarray` conforms to the shape (n_examples, n_features). The second `np.ndarray` conforms to the shape (n_examples, n_labels). n_examples refers to the number of examples in the batch (i.e., 32), n_features refers to the number of features for each example (784), and n_labels refers to the number of output labels (10).

1.4. Key Functions

The following functions were implemented in `nn.py`:

- `init_model(n_input, n_hidden, n_output) -> Model`

Instantiates a `Model` dataclass with normally distributed weights and biases (μ=0.0, σ=0.01).

- `train(batch: tuple[np.ndarray, np.ndarray], m: Model, mm: ModelMetrics, lr: float)`

Performs a training step over the given batch. Internally it performs a forward pass, computes softmax over the network outputs, performs backpropagation and updates the weights/biases.

This description is purposefully brief, for an in-depth explanation of the forward and backward pass computations, please refer to Appendix 4 Task 2 planning.ipynb.

- `eval(batch: tuple[np.ndarray, np.ndarray], m: Model, mm: ModelMetrics)`

Performs a forward pass using the network weights/biases followed by a softmax. Identical to the first two steps of `train`.

- `sigmoid(x) -> np.float64`

Implements the sigmoid activation function: $\sigma(x) = \frac{1}{1+e^{-x}}$.

- `one_hot(y: np.ndarray, n_classes: int) -> np.ndarray`

Performs one hot encoding of labels, that is, it converts `y` of shape (n_examples) into shape (n_examples, n_labels) where n_labels refers to the number of output labels (10).

- `sep_labels(x: np.ndarray, n_classes: int) -> tuple[np.ndarray, np.ndarray]`

Reshapes data as described in section 1.3. Internally it will call `one_hot`.

- `batch(data: tuple[np.ndarray, np.ndarray], batch_size: int) -> Iterator[tuple[np.ndarray, np.ndarray]]`

Generator function to return batches of size `batch_size` from the given dataset.

- `RunningAverage.update(self, new_value)`

Computes incremental mean ($\mu_{n+1} = \mu_n + \frac{x_{n+1}-\mu_n}{n+1}$).

- `RunningAverage.reset(self)`

Resets incremental mean ($\mu_0 \leftarrow 0$).

- `ModelMetrics.update(self, y: np.ndarray, y_hat: np.ndarray)`

Accepts model predictions and ground truth labels (one hot encoded) and computes loss and accuracy. Loss and accuracy attributes of the `ModelMetrics` class are then updated. For details on loss computation please refer to Appendix 4 Task 2 planning.ipynb.

- `ModelMetrics.reset(self)`

Resets `ModelMetrics` attributes to 0 ($\text{loss} \leftarrow 0, \text{accuracy} \leftarrow 0$).

1.5. Optimisations

All model computations (forward pass and backward pass) are performed in highly efficient NumPy operations. Minibatch stochastic gradient descent is used during training due to its

quicker and higher reliability in model convergence. The implemented NumPy operations are vectorized across the entire minibatch to support quick forward and backward passes.

Labels are one-hot-encoded to simplify computations during backpropagation.

In addition, the NumPy operations used in `nn.py` were recreated using JAX operations in `nn_accelerated.py`. JAX allows all previous NumPy operations to be compiled into highly optimised GPU-compatible machine code using XLA. The accelerated variant of `nn.py` supports GPU vectorisation across a single minibatch, a single epoch and across n epochs. In creating the JAX implementation, previously implemented object-oriented classes such as `ModelMetrics` were modified to be functional to support the JAX tracer, which underpins JIT compilation and automatic differentiation.

Only JAX NumPy utilities and XLA capabilities are utilised. To achieve maximum parity with the NumPy only implementation JAX automatic differentiation utilities were not used. All experiments detailed in Section 2 were performed using the NumPy only version.

Table 2 shows runtime across the various implementations. Per-minibatch GPU acceleration leads to a slower execution time compared to the original NumPy version due to overhead resulting from CPU-GPU transfer operations (tops) (1 min 16 s vs. 9.22 s). Over 30 epochs and a batch size of 32, there will be 112500 ($30 \cdot 1875 \cdot 2$) tops significantly inflating runtime. Per-epoch GPU acceleration reduces tops to only 60. All-epoch GPU acceleration reduces tops to only 2. The tops improvement from Per-minibatch GPU accelerated to Per-epoch GPU accelerated or All-epoch GPU accelerated reduces runtime by a factor of ≈ 50 (76 s vs. 1.57 s). Per-epoch GPU acceleration reduced runtime compared to NumPy by a factor of ≈ 6 (9.22 s vs. 1.57 s).

While the JAX implementation compiles successfully with the configuration outlined in Table 2, modifying some parameters (e.g., increasing hidden neuron count to 128) caused compilation errors when targeting the GPU. Strangely those compilation errors were not present when targeting the CPU. Despite this unresolved compilation issue, the performance gains achieved are considerable.

Table 2 Runtime comparison of various model optimisations for a fully connected network (784, 30, 10) over 30 epochs with a learning rate (η) of 3.0 and a mini-batch size of 32.

Training Approach	Time
NumPy only version[1]	9.22 s ± 82.4 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)
All-epoch GPU accelerated[2]	1.57 s ± 30.7 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)
Per-epoch GPU accelerated[2]	1.54 s ± 26.3 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)
Per-minibatch GPU accelerated[2]	1min 16s ± 627 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)

[1] Benchmarking code available in `benchmarking-nn.ipynb`. Please see Appendix 6 Task 2 benchmarking-nn.ipynb.

[2] Benchmarking code available in `benchmarking-nn-accelerated.ipynb`. Please see Appendix 7 Task 2 benchmarking-nn-accelerated.ipynb.

1.6. Screenshots Showing Program Execution

While the model implementation supports execution via command line invocation, Jupyter notebooks were used for conducting experiments and graphing. Jupyter notebooks may be found in the following appendices:

- Appendix 4 Task 2 planning.ipynb
- Appendix 5 Task 2 running_nn.ipynb
- Appendix 6 Task 2 benchmarking-nn.ipynb
- Appendix 7 Task 2 benchmarking-nn-accelerated.ipynb

- Appendix 8 Task 2 hyperparameter-tuning.ipynb

Figure 3 shows command line invocation.

```
(learning-ml) roman@GreenO11:~/learning-ml/2802ict-assignments/assignment_02/nn$ python nn.py 784 30 10 /home/roman/lear
ning-ml/2802ict-assignments/assignment_02/nn/fashion-mnist_train.csv.gz /home/roman/learning-ml/2802ict-assignments/assi
gnment_02/nn/fashion-mnist_test.csv.gz
(60000, 10) (60000, 784)
(10000, 10) (10000, 784)
l=1.8074214715232249 acc=0.3844666666666663
l=1.0245803091552568 acc=0.6463166666666679
l=0.7782042919460797 acc=0.7272000000000003
l=0.6685491087514096 acc=0.7609333333333334
l=0.6051889668320375 acc=0.7865000000000002
l=0.5620549686045951 acc=0.8051166666666671
l=0.5305317091122683 acc=0.8174333333333338
l=0.5067997683452904 acc=0.8253333333333333
```

Figure 3 Screenshot showing `nn.py` command line invocation.

2. Experiments

2.1. Base Case

The baseline model was trained over 30 epochs in 13.4 seconds with a learning rate of 3 and a batch size of 20.

Figure 4 illustrates the learning progression of the model. Both the training and validation accuracy increase rapidly within the first 5 epochs, indicating that the model is effectively learning underlying patterns in the data. After the initial phase, the curves begin to plateau.

Following the initial rapid progression, the training accuracy shows a relatively smooth, monotonic increase, eventually reaching approximately 86%. In contrast, the validation accuracy (blue line) exhibits significant oscillations after its initial climb, fluctuating between roughly 80% and 85%. This volatility suggests that the chosen learning rate of 3.0 is quite high for a mini-batch size of 20. A high learning rate may cause model weights to "overshoot" the optimal point in the loss landscape during updates, leading to instability rather than smooth convergence.

Despite the oscillations, there is no significant divergence between the training and validation curves, which suggests that the model is not severely overfitting the training data. The model generalises reasonably well, achieving a peak validation accuracy of 85%.

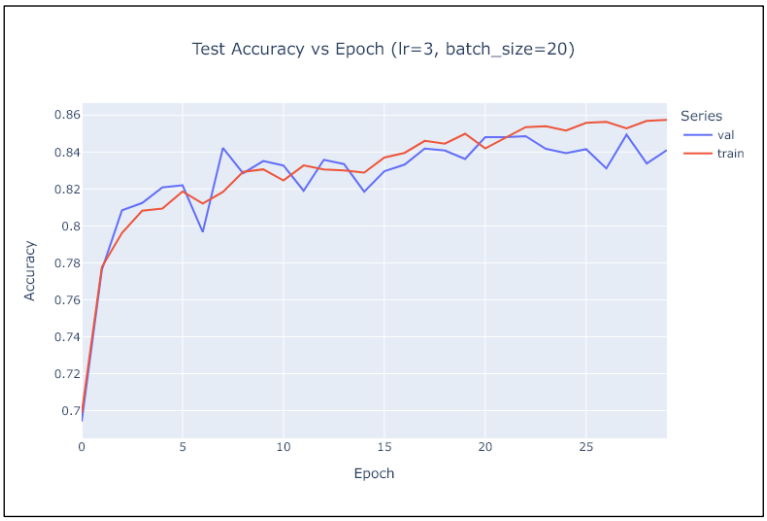


Figure 4 Training and Validation Accuracy vs. Epoch for the Baseline Model. Performance of a fully connected network (784, 30, 10) over 30 epochs with a learning rate (η) of 3.0 and a mini-batch size of 20. The model achieved a maximum validation accuracy of 85%.

2.2. Varying Learning Rate

The model was trained over 30 epochs with a fixed batch size of 20 and varying learning rates of 0.001, 0.01, 1.0, 10, and 100. Total training time was 1 minute 4 seconds.

Figure 5 illustrates the impact of learning rate on model performance. The results can be categorised into three distinct behaviours:

Learning too slowly (η= 0.001)	With a learning rate of 0.001, the model learns very slowly, and the accuracy curve does not plateau after 30 epochs, reaching only 73%. Weight updates are likely too small to allow for convergence across each example.
Optimal learning (η= 1.0, 0.01)	At η = 0.01 or η = 1.0 the learning rate appears optimal for the implemented model. Initial epochs show rapid learning and quick convergence to the highest maximum accuracy of 88%. This learning rate is large enough for fast convergence without causing the instability seen in the baseline model (η=3.0).
Learning too fast (η= 10, 100)	At η = 10 and η = 100, the learning rate is high enough that weight updates overshoot the minimum of the loss function, causing divergence. Accuracy flatlines at a very low level (20% for η=10 and 10% for η=100).

A learning rate of 1.0 provided the best performance, achieving a maximum accuracy of 88%.

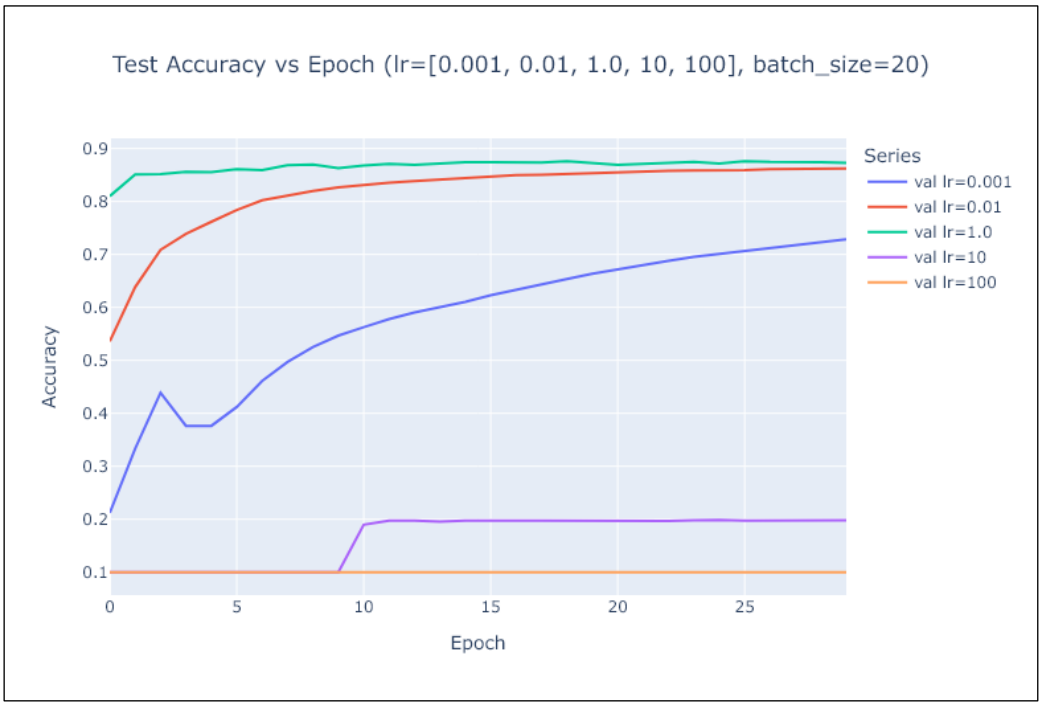


Figure 5 Impact of Learning Rate on Validation Accuracy. Performance of a fully connected network (784, 30, 10) over 30 epochs with a fixed mini-batch size of 20, across five different learning rates (η) (0.001, 0.01, 1.0, 10, 100). The results show a clear optimal range for the learning rate, with η=1.0 achieving the highest accuracy of 88%.

2.3. Varying Batch Size

The model was trained over 30 epochs with a fixed learning rate of 3.0 and varying minibatch sizes of 1, 5, 20, 100 and 300. Total training time was 3 minutes 59 seconds.

Figure 6 illustrates the impact of mini-batch size on model performance.

Smaller batch sizes introduce more noise into the gradient estimate for each weight update. With a batch size of 1 (Stochastic Gradient Descent), the updates are extremely noisy, and combined with a high learning rate, the model fails to converge, flatlining at 10% accuracy. As the batch size increases to 5 and 20, the model learns, but the accuracy curves are highly volatile. The noise in the gradient causes the learning process to be unstable.

As the mini-batch size increases, the gradient estimate for each update becomes more accurate (i.e., closer to the true gradient of the entire training set). This reduction in variance has a stabilising effect on training. The curves for 100 and 300 are much smoother and converge quickly to high accuracy levels. The larger batches effectively average out the noise, allowing the high learning rate to take confident steps toward the minimum.

The highest maximum accuracy of 88% was achieved with a mini-batch size of 300. Regarding computational speed, smaller batch sizes require more weight updates per epoch, potentially increasing training time. Therefore, the training with mini-batch size = 1 was the slowest computationally. This experiment demonstrates that for a given learning rate, there is an optimal range of batch sizes, and larger batches can help stabilise training.

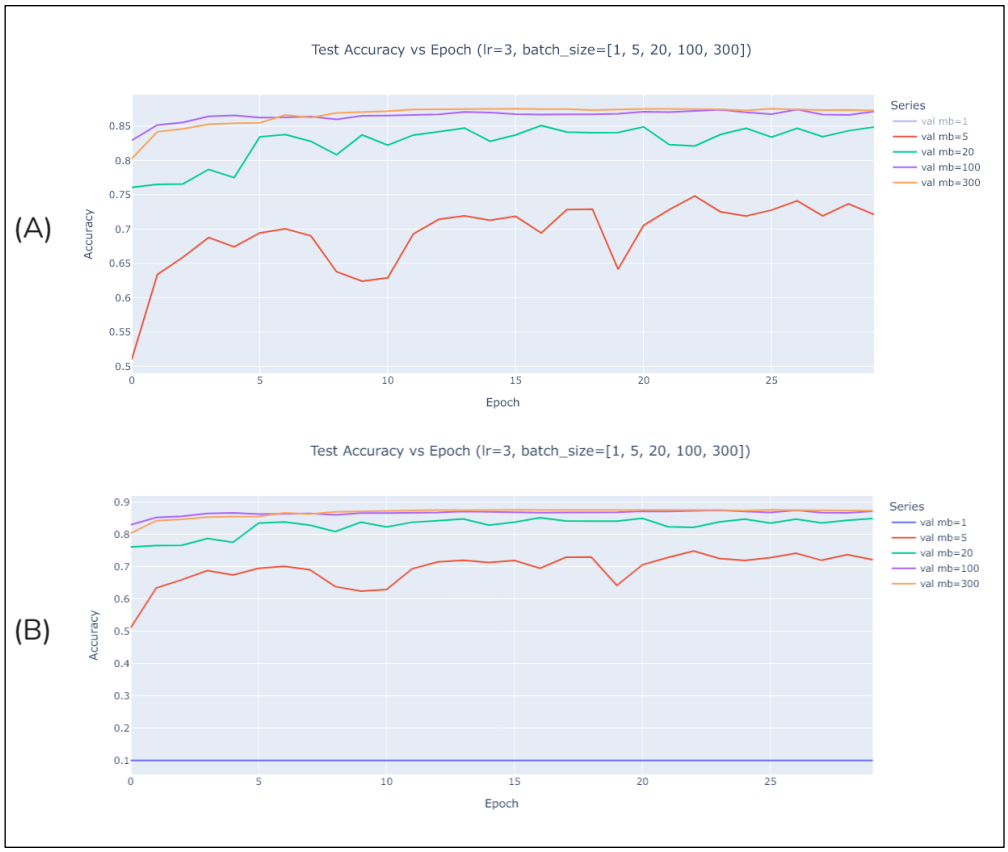


Figure 6 Impact of Minibatch size on Validation Accuracy. Performance of a fully connected network (784, 30, 10) over 30 epochs with a fixed learning rate (η=3.0), across five different minibatch sizes. Graph A shows performance across sizes 5, 20, 100 and 300. Graph B shows performance across sizes 1, 5, 20, 100, 300. Larger mini-batch sizes lead to more stable learning and higher final accuracy, with a size of 300 achieving the best result of 88%.

2.4. Hyperparameter Tuning

Based on the results from Sections 2.2 and 2.3, the identified learning rates and batch sizes were used as heuristics to guide the selection of hyperparameters for the final model.

Ultimately the model was trained over 1 min 4 seconds with a learning rate of 0.1, batch size of 128, epoch count of 60 and hidden neuron count of 128. A maximum validation accuracy of 88% was achieved. Figure 7 shows per epoch model validation and training accuracy. Model performance rapidly increases in the first 5 epochs before plateauing. At the 20-epoch mark, the training and validation curves cross, potentially suggesting overtraining.

Similar conclusions can be drawn from the model loss curves shown in Figure 8.

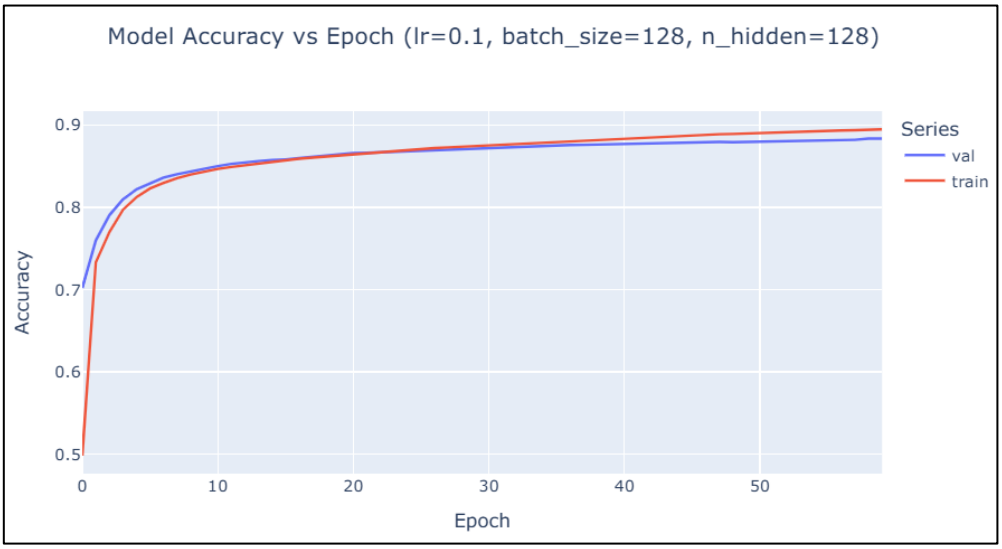


Figure 7 Training and Validation Accuracy vs. Epoch for the finetuned model. Performance of a fully connected network (784, 128, 10) over 30 epochs with a learning rate (η) of 0.1 and a mini-batch size of 128. The model achieved a maximum validation accuracy of 88%.

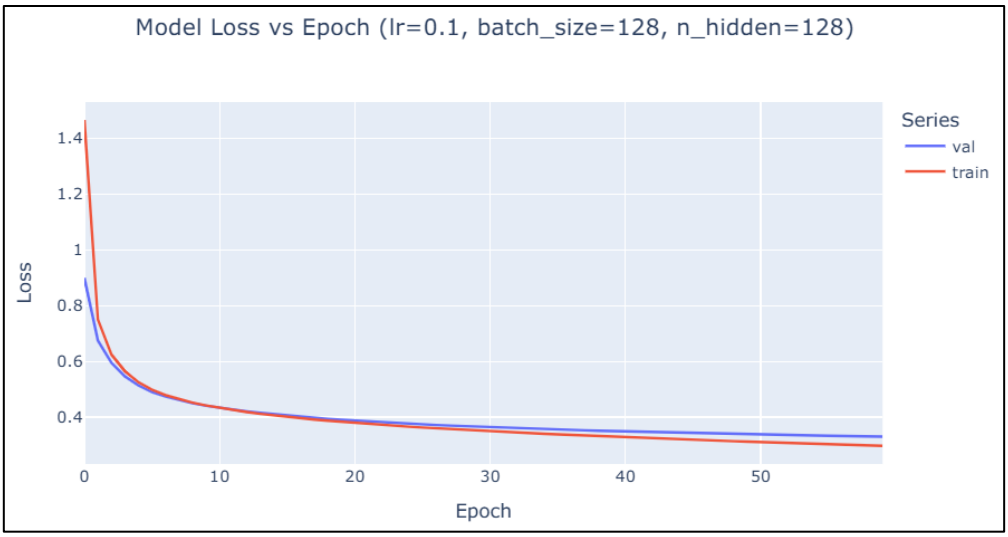


Figure 8 Training and Validation Loss vs. Epoch for the finetuned model. Performance of a fully connected network (784, 128, 10) over 30 epochs with a learning rate (η) of 0.1 and a mini-batch size of 128. The model achieved a minimum validation loss of 0.33.

3. Conclusion

A two-layer fully connected neural network was successfully implemented from scratch using NumPy and trained with the backpropagation algorithm on the Fashion-MNIST dataset. Systematic experimentation revealed the critical impact of hyperparameters on model performance. An optimal learning rate of 1.0 was identified, which provided rapid yet stable convergence, while larger mini-batch sizes (e.g., 300) were shown to mitigate the instability caused by high learning rates. The final optimized model, resulting from hyperparameter tuning, achieved a peak validation accuracy of 88%. Furthermore, a JAX-based version of the network was developed to leverage GPU acceleration, demonstrating a reduction in training time.

Appendix 1 Task 1 Source Code

1. decision_tree.py

```
import sys
import polars as pl
import math
from dataclasses import dataclass, field
from typing import Any, Literal, Union, cast

Status = Literal["unacc", "acc", "good", "vgood"]

CAR_COLUMNS = {
    "buying": pl.Enum(["low", "med", "high", "vhigh"]),
    "maint": pl.Enum(["low", "med", "high", "vhigh"]),
    "doors": pl.Enum(["2", "3", "4", "5more"]),
    "persons": pl.Enum(["2", "3", "4", "more"]),
    "lug_boot": pl.Enum(["small", "med", "big"]),
    "safety": pl.Enum(["low", "med", "high"]),
    "class": pl.Enum(["unacc", "acc", "good", "vgood"]),
}
OUTCOME_CLASS = "class"

@dataclass
class InteriorNode:
    attribute: str
    plurality_prediction: Status # used when processing example and value for the attribute
is not in the decision_tree
    decision_tree: 'dict[str, Node]' = field(default_factory=dict)

@dataclass
class LeafNode:
    value: Status

type Node = Union[InteriorNode, LeafNode]

def entropy(examples: pl.DataFrame) -> float:
    outcomes = examples[OUTCOME_CLASS].unique()
    total = examples.height
    chances = list(map(lambda k: examples.filter(pl.col(OUTCOME_CLASS) == k).height /
total, outcomes))
    return -sum(p * math.log(p, 2) for p in chances if p > 0)

def remainder(examples: pl.DataFrame, attribute: str) -> float:
    total = examples.height
    final = 0
    ## Weighted average of entropy when filtered by each attribute value
    for attr in examples[attribute].unique():
        subset_examples = examples.filter(pl.col(attribute) == attr)
        final += (subset_examples.height/total)*entropy(subset_examples)
    return final

def importance(examples: pl.DataFrame, attribute: str):
    ## The size of the reduction in entropy
    return entropy(examples) - remainder(examples, attribute)

def plurality_value(examples: pl.DataFrame) -> Status:
    mode_value = examples[OUTCOME_CLASS].mode().first()
    return cast(Status, mode_value) # polars return types make me sad

def ID3(examples: pl.DataFrame, attributes: set[str], parent_examples: pl.DataFrame) ->
Node:
    if examples.height == 0:
        return LeafNode(plurality_value(parent_examples))
    elif examples[OUTCOME_CLASS].n_unique() == 1:
        return LeafNode(plurality_value(examples))
```

```python
    elif len(attributes) == 0:
        return LeafNode(plurality_value(examples))
    else:
        ## Find best attribute

        # decision_attribute = max(attributes, key=lambda x: importance(examples, x))

        current_entropy = entropy(examples)
        decision_attribute = max(attributes, key=lambda x: current_entropy -
remainder(examples, x))
        # this is slightly quicker than using the importance function as it
        # avoids recomputing entropy for examples over and over

        ## Construct tree
        tree = InteriorNode(decision_attribute, plurality_value(examples))

        for value in examples[decision_attribute].unique():
            filtered_examples = examples.filter(pl.col(decision_attribute) == value)
            subtree = ID3(filtered_examples, attributes - {decision_attribute}, examples)
            tree.decision_tree[value] = subtree

        return tree

def classify(row: dict[str, Any], node: Node) -> Status:
    if isinstance(node, LeafNode):
        return node.value
    elif isinstance(node, InteriorNode):
        value = row[node.attribute]
        if value not in node.decision_tree: return node.plurality_prediction
        else: return classify(row, node.decision_tree[value])

def pretty_print_tree(node: Node, indent: str = ""):
    if isinstance(node, LeafNode):
        print(f"{indent}Leaf: {node.value}")
    elif isinstance(node, InteriorNode):
        print(f"{indent}Attribute: {node.attribute}")
        for value, subtree in node.decision_tree.items():
            print(f"{indent}  Value: {value}")
            pretty_print_tree(subtree, indent + "    ")

def main():
    dataset_name = sys.argv[1]

    df_csv = pl.read_csv(dataset_name, schema_overrides=CAR_COLUMNS)

    tree = ID3(df_csv, set(df_csv.columns) - {OUTCOME_CLASS}, df_csv)

    pretty_print_tree(tree)

if __name__ == "__main__":
    main()
```

2. helpers.py

```python
from dataclasses import dataclass, field
from functools import reduce

@dataclass
class Metric:
    TP: int = 0
    FP: int = 0
    TN: int = 0
    FN: int = 0

    def total(self) -> int: ## equal to the total number of samples in the test set
        return self.TP + self.FP + self.TN + self.FN
```

```python
    def accuracy(self) -> float:
        if self.total() == 0: return 0.0
        return (self.TP + self.TN) / self.total()

    def precision(self) -> float:
        denominator = self.TP + self.FP
        if denominator == 0: return 0.0
        return self.TP / denominator

    def recall(self) -> float:
        denominator = self.TP + self.FN
        if denominator == 0: return 0.0
        return self.TP / denominator

    def f1_score(self) -> float:
        prec = self.precision()
        recall = self.recall()
        denominator = prec + recall
        if denominator == 0: return 0.0
        return 2 * prec * recall / denominator

    def support(self) -> int: # the count of this class (ie unacc)
        return self.TP + self.FN

class Metrics():
    def __init__(self, classes: list[str]):
        self.metrics = {i:Metric() for i in classes}

    def update(self, model_prediction: str, ground_truth: str):
        for cls in self.metrics:
            if model_prediction == cls and cls == ground_truth: # model predicted class and
was class was right
                self.metrics[cls].TP += 1
            elif model_prediction != cls and cls != ground_truth: # model did not predict
class and class was not right
                self.metrics[cls].TN += 1
            elif model_prediction == cls and model_prediction != ground_truth: # model
predicted class and class was not right
                self.metrics[cls].FP += 1
            elif model_prediction != cls and cls == ground_truth: # model did not predict
class and class was right
                self.metrics[cls].FN += 1

    def print_summary(self):
        print(f"{'':<20} {'precision':<15} {'recall':<15} {'f1-score':<15}
{'support':<15}")
        print("-" * 80)

        for key, value in self.metrics.items():
            print(f"{key:<20} {value.precision():<15.4f} {value.recall():<15.4f}
{value.f1_score():<15.4f} {value.support():<15}")

        print("-" * 80)

        avg_results = self.average_results()
        total = self.total()

        print(f"{'macro avg':<20} {avg_results['macro_precision']:<15.4f}
{avg_results['macro_recall']:<15.4f} {avg_results['macro_f1_score']:<15.4f} {total:<15}")
        print(f"{'weighted avg':<20} {avg_results['weighted_precision']:<15.4f}
{avg_results['weighted_recall']:<15.4f} {avg_results['weighted_f1_score']:<15.4f}
{total:<15}")

        print("-" * 80)
```

```
        print()

        print(f"{"accuracy":<20} {self.accuracy():.4f}")

    def total(self) -> int:
        return next(iter(self.metrics.values())).total()
        # this also works but would be slower:
        # return sum(map(lambda k: k.support(), self.metrics.values()))

    def accuracy(self) -> float:
        return sum(map(lambda k: k.TP, self.metrics.values())) / self.total()

    def average_results(self) -> dict[str, float]:
        scores: dict[str, float] = {}

        total = self.total()

        scores["macro_precision"] = sum([k.precision() for k in
self.metrics.values()])/len(self.metrics)
        scores["weighted_precision"] = sum([k.precision()*k.support() for k in
self.metrics.values()])/total

        scores["macro_recall"] = sum([k.recall() for k in
self.metrics.values()])/len(self.metrics)
        scores["weighted_recall"] = sum([k.recall()*k.support() for k in
self.metrics.values()])/total

        scores["macro_f1_score"] = sum([k.f1_score() for k in
self.metrics.values()])/len(self.metrics)
        scores["weighted_f1_score"] = sum([k.f1_score()*k.support() for k in
self.metrics.values()])/total

        return scores
```

Appendix 2 Task 2 Source Code

1. nn.py

```
import numpy as np
import sys
from typing import Iterator
from dataclasses import dataclass, field

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def one_hot(y: np.ndarray, n_classes: int) -> np.ndarray:
    y_onehot = np.zeros((y.shape[0], n_classes))
    for i, label in enumerate(y):
        y_onehot[i, int(label)] = 1
    return y_onehot

# I'm just more used to working with training data in this format
# normalise and one hot encode
def sep_labels(x: np.ndarray, n_classes: int) -> tuple[np.ndarray, np.ndarray]:
    labels = one_hot(x[:, 0], n_classes)
    features = x[:, 1:] / 256
    print(labels.shape, features.shape)
    return (features, labels)

def batch(data: tuple[np.ndarray, np.ndarray], batch_size: int) ->
Iterator[tuple[np.ndarray, np.ndarray]]:
    for start in range(0, data[0].shape[0], batch_size):
        end = min(data[0].shape[0], start + batch_size)
        yield (data[0][start:end], data[1][start:end])

@dataclass
class Model:
    W_1: np.ndarray
    B_1: np.ndarray
    W_2: np.ndarray
    B_2: np.ndarray

@dataclass
class RunningAverage:
    value: float = 0.0
    count: int = 0

    def update(self, new_value):
        self.value = (self.value * self.count + new_value)/(self.count+1)
        self.count += 1
    def reset(self):
        self.value = 0.0
        self.count = 0

@dataclass
class ModelMetrics:
    loss: RunningAverage = field(default_factory=RunningAverage)
    accuracy: RunningAverage = field(default_factory=RunningAverage)

    def update(self, y: np.ndarray, y_hat: np.ndarray):
        # loss
        L = y * y_hat
        L = L.sum(axis=1)
        L = -np.log(L)
        L = L.mean()
        self.loss.update(L)

        # accuracy
        pred = y_hat.argmax(axis=1)
```

```
        true = y.argmax(axis=1)
        accuracy = (pred == true).mean()
        self.accuracy.update(accuracy)

    def reset(self):
        self.loss.reset()
        self.accuracy.reset()

def init_model(n_input, n_hidden, n_output) -> Model:
    W_1 = np.random.normal(0, 0.01, (n_hidden, n_input))
    B_1 = np.random.normal(0, 0.01, n_hidden)
    W_2 = np.random.normal(0, 0.01, (n_output, n_hidden))
    B_2 = np.random.normal(0, 0.01, n_output)
    return Model(W_1, B_1, W_2, B_2)

def train(batch: tuple[np.ndarray, np.ndarray], m: Model, mm: ModelMetrics, lr: float):
    # the logic behind this function can be found in planning.ipynb/planning.pdf

    n_examples = batch[0].shape[0]
    X = batch[0]
    y = batch[1]

    # forward pass
    Z_1 = X@m.W_1.T+m.B_1
    H_1 = sigmoid(Z_1)
    Z_2 = H_1@m.W_2.T+m.B_2

    # softmax
    adj_Z_2 = np.exp(Z_2 - Z_2.max(axis=1)[:, np.newaxis])
    adj_Z_2_sum = np.sum(adj_Z_2, 1)[:, np.newaxis]
    y_hat = adj_Z_2 / adj_Z_2_sum

    # backprop layer 2
    Delta_2 = y_hat - y
    grad_W2 = (Delta_2.T @ H_1) / n_examples
    grad_B2 = Delta_2.mean(axis=0)

    # backprop layer 1
    S_1 = sigmoid(Z_1)
    Delta_1 = (Delta_2 @ m.W_2) * S_1 * (1 - S_1)
    grad_W1 = (Delta_1.T @ X) / n_examples
    grad_B1 = Delta_1.mean(axis=0)

    # update weights
    m.W_2 = m.W_2 - lr * grad_W2
    m.B_2 = m.B_2 - lr * grad_B2
    m.W_1 = m.W_1 - lr * grad_W1
    m.B_1 = m.B_1 - lr * grad_B1

    mm.update(y, y_hat)

    return

def eval(batch: tuple[np.ndarray, np.ndarray], m: Model, mm: ModelMetrics):
    X = batch[0]
    y = batch[1]

    # forward pass
    Z_1 = X@m.W_1.T+m.B_1
    H_1 = sigmoid(Z_1)
    Z_2 = H_1@m.W_2.T+m.B_2

    # softmax
    adj_Z_2 = np.exp(Z_2 - Z_2.max(axis=1)[:, np.newaxis])
    adj_Z_2_sum = np.sum(adj_Z_2, 1)[:, np.newaxis]
    y_hat = adj_Z_2 / adj_Z_2_sum
```

```
    mm.update(y, y_hat)

    return


if __name__ == "__main__":

    n_input = int(sys.argv[1])
    n_hidden = int(sys.argv[2])
    n_output = int(sys.argv[3])

    classes = 10
    lr = 0.02
    epochs = 100

    d_train = sep_labels(np.loadtxt(sys.argv[4], delimiter=',', skiprows=1), classes) #
(60000, 785) first col is label
    d_test = sep_labels(np.loadtxt(sys.argv[5], delimiter=',', skiprows=1), classes) #
(10000, 785) first col is label

    model = init_model(n_input, n_hidden, n_output)

    metrics = ModelMetrics()

    for i in range(epochs):
        loss = 0
        # mb mini batch
        for mb in batch(d_train, 32):
            train(mb, model, metrics, lr)
        print(f"l={metrics.loss.value} acc={metrics.accuracy.value}")
        metrics.reset()
```

2. nn_accelerated.py

```
import numpy as np
import sys
from typing import Iterator
from dataclasses import dataclass, field

import jax
from jax import numpy as jnp
from jax import random
from functools import partial

def sigmoid(x):
    return 1 / (1 + jnp.exp(-x))

def one_hot(y: np.ndarray, n_classes: int) -> np.ndarray:
    y_onehot = np.zeros((y.shape[0], n_classes))
    for i, label in enumerate(y):
        y_onehot[i, int(label)] = 1
    return y_onehot

# I'm just more used to working with training data in this format
# normalise and one hot encode
def sep_labels(x: np.ndarray, n_classes: int) -> tuple[jax.Array, jax.Array]:
    labels = one_hot(x[:, 0], n_classes)
    features = x[:, 1:] / 256
    return (jnp.array(features), jnp.array(labels))

def batch(data: tuple[jax.Array, jax.Array], batch_size: int) -> Iterator[tuple[jax.Array,
jax.Array]]:
    for start in range(0, data[0].shape[0], batch_size):
        end = min(data[0].shape[0], start + batch_size)
        yield (data[0][start:end], data[1][start:end])
```

```python
def to_device_batches(x_np, y_np, batch_size): # will drop any remainder
    x = jnp.array(x_np, dtype=jnp.float32)
    y = jnp.array(y_np, dtype=jnp.float32)
    n = (x.shape[0] // batch_size) * batch_size
    x = x[:n]
    y = y[:n]
    xb = x.reshape(n // batch_size, batch_size, -1)   # (B, N, D)
    yb = y.reshape(n // batch_size, batch_size, -1)   # (B, N, C)
    return xb, yb

@jax.tree_util.register_dataclass
@dataclass
class Model:
    W_1: jax.Array
    B_1: jax.Array
    W_2: jax.Array
    B_2: jax.Array

@jax.tree_util.register_dataclass
@dataclass
class RunningAverage:
    value: jax.Array = field(default_factory=lambda: jnp.array(0.0, jnp.float32))
    count: jax.Array = field(default_factory=lambda: jnp.array(0, jnp.int32))

    def update(self, new_value):
        n_value = (self.value * self.count + new_value)/(self.count+1)
        n_count = self.count + 1
        return RunningAverage(n_value, n_count)
    def reset(self):
        return RunningAverage(jnp.array(0.0, jnp.float32), jnp.array(0, jnp.int32))

@jax.tree_util.register_dataclass
@dataclass
class ModelMetrics:
    loss: RunningAverage = field(default_factory=RunningAverage)
    accuracy: RunningAverage = field(default_factory=RunningAverage)

    def update(self, y: jax.Array, y_hat: jax.Array):
        # loss
        L = y * y_hat
        L = L.sum(axis=1)
        L = -jnp.log(L + 1e-8)
        L = L.mean()
        loss = self.loss.update(L)

        # accuracy
        pred = y_hat.argmax(axis=1)
        true = y.argmax(axis=1)
        accuracy = (pred == true).mean()
        accuracy = self.accuracy.update(accuracy)

        return ModelMetrics(loss, accuracy)

    def reset(self):
        return ModelMetrics(RunningAverage(), RunningAverage())

def init_model(key, n_input, n_hidden, n_output) -> Model:
    key1, key2, key3, key4 = random.split(key, 4)
    W_1 = jax.random.normal(key1, (n_hidden, n_input)) * 0.01
    B_1 = jax.random.normal(key2, (n_hidden)) * 0.01
    W_2 = jax.random.normal(key3, (n_output, n_hidden)) * 0.01
    B_2 = jax.random.normal(key4, (n_output)) * 0.01
    return Model(W_1, B_1, W_2, B_2)
```

```python
@jax.jit
def train_step(batch: tuple[jax.Array, jax.Array], m: Model, mm: ModelMetrics, lr: float):
    print("compiling train_step")
    # the logic behind this function can be found in planning.ipynb/planning.pdf
    n_examples = batch[0].shape[0]
    X = batch[0]
    y = batch[1]

    # forward pass
    Z_1 = X@m.W_1.T+m.B_1
    H_1 = sigmoid(Z_1)
    Z_2 = H_1@m.W_2.T+m.B_2

    # softmax
    adj_Z_2 = jnp.exp(Z_2 - Z_2.max(axis=1)[:, jnp.newaxis])
    adj_Z_2_sum = jnp.sum(adj_Z_2, 1)[:, jnp.newaxis]
    y_hat = adj_Z_2 / adj_Z_2_sum

    # backprop layer 2
    Delta_2 = y_hat - y
    grad_W2 = (Delta_2.T @ H_1) / n_examples
    grad_B2 = Delta_2.mean(axis=0)

    # backprop layer 1
    S_1 = sigmoid(Z_1)
    Delta_1 = (Delta_2 @ m.W_2) * S_1 * (1 - S_1)
    grad_W1 = (Delta_1.T @ X) / n_examples
    grad_B1 = Delta_1.mean(axis=0)

    # update weights
    W_2 = m.W_2 - lr * grad_W2
    B_2 = m.B_2 - lr * grad_B2
    W_1 = m.W_1 - lr * grad_W1
    B_1 = m.B_1 - lr * grad_B1
    m = Model(W_1, B_1, W_2, B_2)

    mm = mm.update(y, y_hat)

    return (m, mm)

@jax.jit
def eval(batch: tuple[jax.Array, jax.Array], m: Model, mm: ModelMetrics):
    print("compiling eval")
    X = batch[0]
    y = batch[1]

    # forward pass
    Z_1 = X@m.W_1.T+m.B_1
    H_1 = sigmoid(Z_1)
    Z_2 = H_1@m.W_2.T+m.B_2

    # softmax
    adj_Z_2 = jnp.exp(Z_2 - Z_2.max(axis=1)[:, jnp.newaxis])
    adj_Z_2_sum = jnp.sum(adj_Z_2, 1)[:, jnp.newaxis]
    y_hat = adj_Z_2 / adj_Z_2_sum

    mm = mm.update(y, y_hat)

    return mm

@jax.jit
def run_all_evals(m, mm, xb, yb):
    print("compiling run_evals")
    def body(carry, t):
        mm = carry
        x = xb[t]
```

```
        y = yb[t]
        mm = eval((x, y), m, mm)
        return mm, None
    mm, _ = jax.lax.scan(body, mm, jnp.arange(xb.shape[0]))
    return mm

@jax.jit
def run_epoch(m, mm, xb, yb, lr):
    print("compiling run_epoch")
    def body(carry, t):
        m, mm = carry
        x = xb[t]
        y = yb[t]
        m, mm = train_step((x, y), m, mm, lr)
        return (m, mm), None
    (m, mm), _ = jax.lax.scan(body, (m, mm), jnp.arange(xb.shape[0]))
    return m, mm

@partial(jax.jit, static_argnames=("epochs"))
def run_all_epochs(m, mm, xb, yb, epochs, lr):
    print("compiling run_all_epochs")
    def body(carry, t):
        m, mm = carry
        m, mm = run_epoch(m, mm, xb, yb, lr)
        return (m, mm), None
    (m, mm), _ = jax.lax.scan(body, (m, mm), jnp.arange(epochs))
    return m, mm

if __name__ == "__main__":

    n_input = int(sys.argv[1])
    n_hidden = int(sys.argv[2])
    n_output = int(sys.argv[3])

    classes = 10
    lr = 0.02
    epochs = 100

    d_train = sep_labels(np.loadtxt(sys.argv[4], delimiter=',', skiprows=1), classes) #
(60000, 785) first col is label
    d_test = sep_labels(np.loadtxt(sys.argv[5], delimiter=',', skiprows=1), classes) #
(10000, 785) first col is label

    key = random.key(0)
    model = init_model(key, n_input, n_hidden, n_output)

    metrics = ModelMetrics()

    batched_d_train = to_device_batches(d_train[0], d_train[1], 32)
    print(batched_d_train[0].shape, batched_d_train[1].shape)

    # model, metrics = run_all_epochs(model, metrics, batched_d_train[0],
batched_d_train[1], epochs, lr)
    # print(f"l={metrics.loss.value} acc={metrics.accuracy.value}")
    # metrics = metrics.reset()


    for i in range(epochs):
        loss = 0
        # mb mini batch
        model, metrics = run_epoch(model, metrics, batched_d_train[0], batched_d_train[1],
lr)
        print(f"l={metrics.loss.value} acc={metrics.accuracy.value}")
        metrics = metrics.reset()
```

Appendix 3 Task 1 eval.ipynb

Evaluation Notebook for Intelligent Systems Assignment 2

In [17]:
```
from decision_tree import ID3, CAR_COLUMNS, OUTCOME_CLASS, classify
import polars as pl
import numpy as np
from helpers import Metric, Metrics
from tqdm import tqdm # for progress bar
import plotly.express as px
import plotly
plotly.offline.init_notebook_mode()
```

In [6]:
```
df_csv = pl.read_csv("./dt-car.csv", schema_overrides=CAR_COLUMNS)
```

In [7]:
```
def partition_dateset(df: pl.DataFrame, partition_percent):
    temp_df = df_csv.with_columns(pl.lit(np.random.rand(df_csv.height)).alias("random_no"))
    train = temp_df.filter(pl.col("random_no") <= partition_percent).drop("random_no")
    test = temp_df.filter(pl.col("random_no") > partition_percent).drop("random_no")
    return (train, test)

train, test = partition_dateset(df_csv, 0.8)

print(f"test = {test.height}, train = {train.height}")
```

```
test = 348, train = 1380
```

In [8]:
```
tree = ID3(train, set(train.columns) - {OUTCOME_CLASS}, train)
```

$$\text{Accuracy} = \frac{TP+TN}{FN+FP+TP+TN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

In [9]:
```
classes = test[OUTCOME_CLASS].unique()
m = Metrics(list(classes))

for row in test.rows(named=True):
    model_prediction = classify(row, tree)
    ground_truth = row["class"]

    m.update(model_prediction, ground_truth)
```

In [10]:
```
m.print_summary()
```

```
                    precision       recall          f1-score        support
--------------------------------------------------------------------------------
unacc               0.9671          0.9592          0.9631          245
acc                 0.8767          0.8205          0.8477          78
good                0.6471          0.9167          0.7586          12
vgood               0.7333          0.8462          0.7857          13
--------------------------------------------------------------------------------
macro avg           0.8060          0.8856          0.8388          348
weighted avg        0.9271          0.9224          0.9236          348
--------------------------------------------------------------------------------

accuracy            0.9224
```

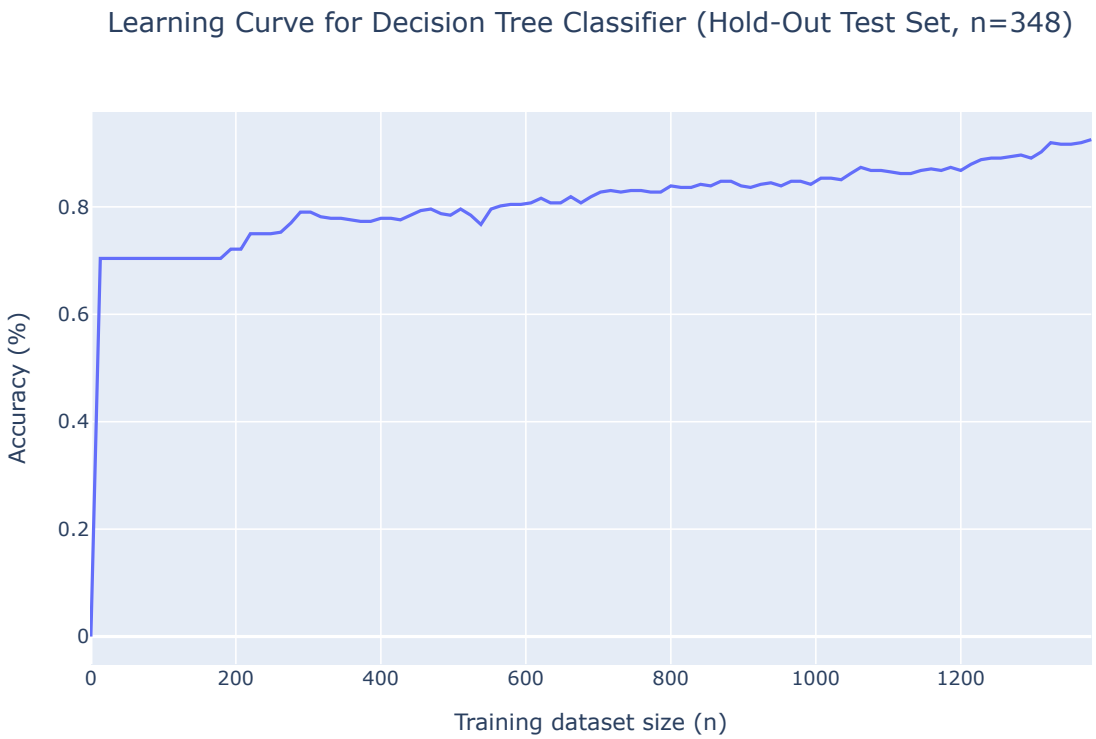
In [12]:
```
partitions = np.arange(0, 1.01, 0.01)
training_sizes = [int(i * train.height) for i in partitions]

accuracies = []

pbar = tqdm(training_sizes)

for size in pbar:
    training_partition = train[0:size]
    tree = ID3(training_partition, set(training_partition.columns) - {OUTCOME_CLASS},
```

```
training_partition)

    correct = 0
    for row in test.rows(named=True):
        model_prediction = classify(row, tree)
        ground_truth = row["class"]
        if model_prediction == ground_truth: correct += 1
    acc = correct/test.height

    pbar.set_description(f"Tree for {size} complete (acc={acc:.2f}%)")

    accuracies.append(acc)
```

```
Tree for 1380 complete (acc=0.93%): 100%|██████████| 101/101 [00:48<00:00,  2.07it/s]
```

In [18]:

```
import plotly.express as px

fig = px.line(x=training_sizes, y=accuracies, labels={"x":"Training dataset size (n)", "y":"Accuracy
(%)"}, title=f"Learning Curve for Decision Tree Classifier (Hold-Out Test Set, n={test.height})")
fig.update_layout(title={'x': 0.5, 'xanchor': 'center'})
fig.show()
```



Appendix 4 Task 2 planning.ipynb

Planning

Jupyter notebook just for planning out matrix operations and making sure I have the right matrix dimensions, order, etc.

Architecture

- 1 input layer; 1 hidden layer; 1 output layer.
  - 784 input nodes (each pixel in the image) (NInput)
  - 30 hidden nodes (NHidden)
  - 10 output nodes (NOutput)
- Activation function: sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$ where $\sigma(x)' = \sigma(x) \cdot (1 - \sigma(x))$

In [1]:
```
import numpy as np
import random
```

In [2]:
```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

Layer Equations

First as single example:

Layer 1:

- `(n_hidden) = (n_hidden, n_features) x (n_features) + (n_hidden)`
- $z_1 = W_1 x + b_1$
- `elementwise`
- $h_1 = \sigma(z_1)$

Layer 2:

- $z_2 = W_2 h_1 + b_2$
- No need for activation

In [3]:
```
n_input = 784 # (n_features)
n_hidden = 30
n_output = 10
learning_rate = 1e-2

x = np.random.normal(0, 0.01, 784)
W_1 = np.random.normal(0, 0.01, (n_hidden, n_input))
b_1 = np.random.normal(0, 0.01, n_hidden)

print(x.shape, W_1.shape, b_1.shape)

z_1 = W_1@x+b_1

print(z_1.shape)
```

```
(784,) (30, 784) (30,)
(30,)
```

Now as vectorised minibatch:

Layer 1:

- W_1 shape is `(n_hidden, n_features)`
- x `(n_features)` is now X `(n_examples, n_features)`
- In order for the dimensions to match, we can transpose W_1 then pre-multiply by X, yielding:
  - `(n_examples, n_features) x (n_hidden, n_features).T = (n_examples, n_features) x (n_features, n_hidden)`

- `(n_examples, n_hidden)`
- $Z_1 = XW_1^\top + B_1$
- $H_1 = \sigma(Z_1)$

Layer 2:

- W_2 shape is `(n_output, n_hidden)`
- h_1 `(n_hidden)` is now H_1 `(n_examples, n_hidden)`
- We can apply the same logic from layer 1 for layer 2, yielding:
  - `(n_examples, n_output)`
  - $Z_2 = H_1 W_2^\top + B_2$

In [4]:
```
n_examples = 32

n_input = 784 # (n_features)
n_hidden = 30
n_output = 10

X = np.random.normal(0, 0.01, (n_examples, 784))
print(f'X {X.shape}')

W_1 = np.random.normal(0, 0.01, (n_hidden, n_input))
B_1 = np.random.normal(0, 0.01, n_hidden)
print(f'W_1 {W_1.shape}, B_1 {B_1.shape}')

Z_1 = X@W_1.T+B_1
print(f'Z_1 {Z_1.shape}')

H_1 = sigmoid(Z_1)

W_2 = np.random.normal(0, 0.01, (n_output, n_hidden))
B_2 = np.random.normal(0, 0.01, n_output)
print(f'W_2 {W_2.shape}, B_2 {B_2.shape}')

Z_2 = H_1@W_2.T+B_2
print(f'Z_2 {Z_2.shape}')
```

```
X (32, 784)
W_1 (30, 784), B_1 (30,)
Z_1 (32, 30)
W_2 (10, 30), B_2 (10,)
Z_2 (32, 10)
```

Softmax

Conventionally:

$$\hat{y}_i = \frac{e^{o_i}}{\sum_k e^{o_k}}$$

But due to overflow risk:

$$\bar{o} \stackrel{\text{def}}{=} \max_k o_k$$

$$\hat{y}_i = \frac{e^{o_i - \bar{o}}}{\sum_k e^{o_k - \bar{o}}}$$

In [5]:
```
## ignore block, just for checking broadcasting was working correctly

max_values = Z_2.max(axis=1)

max_values = max_values[:, np.newaxis]

print(max_values[0:2])
Z_2[0,0] = 5
Z_2[1,0] = 10
print(Z_2[0:2,0:3])

adj_Z_2 = Z_2 - max_values
```

```
print(adj_Z_2[0:2,0:3])
print(f'max_values {max_values.shape}')

# (Z_2 - Z_2.max(axis=1)).shape
```

```
[[0.03740673]
 [0.03746558]]
[[5.00000000e+00 1.65467628e-03 1.24560784e-02]
 [1.00000000e+01 1.65894645e-03 1.24045565e-02]]
[[ 4.96259327 -0.03575205 -0.02495065]
 [ 9.96253442 -0.03580663 -0.02506102]]
max_values (32, 1)
```

In [6]:
```
adj_Z_2 = np.exp(Z_2 - Z_2.max(axis=1)[:, np.newaxis])

print(f'adj_Z_2 {adj_Z_2.shape}')

adj_Z_2_sum = np.sum(adj_Z_2, 1)[:, np.newaxis]

print(f'adj_Z_2_sum {adj_Z_2_sum.shape}')

y_hat = adj_Z_2 / adj_Z_2_sum

print(f'y_hat {y_hat.shape}')

print(y_hat.argmax(axis=1))
```

```
adj_Z_2 (32, 10)
adj_Z_2_sum (32, 1)
y_hat (32, 10)
[0 0 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7]
```

Figuring out the loss function

Cross entropy loss over an example is as follows:

$$\ell = -\sum_{i=1}^{q} y_i \cdot \log(\hat{y}_i)$$

But because y_i is one-hot encoded, we can just take the value of the true class:

$$\ell = -\log(\hat{y}_i)$$

Then over the batch we can just sum up our losses:

$$\mathcal{L} = -\sum_{j=1}^{n} \log(\hat{y}_i)$$

In [7]:
```
# Lets generate some random one-hot encodes for testing

y = np.zeros_like(y_hat)
for i in range(y.shape[0]):
    index = random.randint(0,n_output-1)
    y[i, index] = 1

print(y.shape)

L = y * y_hat
L = L.sum(axis=1)
L = -np.log(L)
# For strict MLE should be sum but I think it's more idiomatic to use mean
L = L.mean()

L
```

```
(32, 10)
```

Out[7]:
```
np.float64(2.6271448572554745)
```

Figuring out backprop (loss function)

First part, figuring out how $\mathcal{L}$ changes with respect to Z_2

$$\frac{\partial \mathcal{L}}{\partial Z_2} = \hat{y} - y$$

In [8]:
```
Delta_2 = y_hat - y

print(f'partial_L {Delta_2.shape}')
```

```
partial_L (32, 10)
```

Second part, figuring out how to update W_2

$$W_2 \leftarrow W_2 - \eta \cdot \frac{\partial \mathcal{L}}{\partial W_2} \tag{1}$$

$$W_2 \leftarrow W_2 - \eta \cdot \frac{\partial \mathcal{L}}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2} \tag{2}$$

Then recall $z_2 = W_2 h_1 + b_2;\ Z_2 = H_1 W_2^\top + B_2$ so in rough terms $\frac{\partial z_2}{\partial W_2} = h_1$ and $\frac{\partial Z_2}{\partial W_2} = H_1$

We want:

- W_2 is `(n_output, n_hidden)`

from:

- $\frac{\partial Z_2}{\partial W_2} = H_1$ is `(n_examples, n_hidden)`
- $\frac{\partial \mathcal{L}}{\partial Z_2}$ is `(n_examples, n_output)`

If we transpose $\frac{\partial \mathcal{L}}{\partial Z_2}$ we get `(n_output, n_examples)` so $\frac{\partial \mathcal{L}}{\partial Z_2}^\top \cdot H_1$ will give us `(n_output, n_hidden)`

By convention let's $\Delta_2 \stackrel{\text{def}}{=} \frac{\partial \mathcal{L}}{\partial Z_2}$ and to avoid scaling weight updates with batch size we'll divide by N:

$$W_2 \leftarrow W_2 - \eta \cdot \frac{1}{N} \cdot \Delta_2^\top \cdot H_1$$

In [9]:
```
grad_W2 = (Delta_2.T @ H_1) / n_examples

print(f"grad_W2 {grad_W2.shape} W2 {W_2.shape}")
```

```
grad_W2 (10, 30) W2 (10, 30)
```

Third part, figuring out how to update B_2

$$B_2 \leftarrow B_2 - \eta \cdot \frac{\partial \mathcal{L}}{\partial B_2} \tag{3}$$

$$B_2 \leftarrow B_2 - \eta \cdot \frac{\partial \mathcal{L}}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial B_2} \tag{4}$$

Then recall $z_2 = W_2 h_1 + b_2;\ Z_2 = H_1 W_2^\top + B_2$ so in rough terms $\frac{\partial z_2}{\partial b_2} = 1$ and $\frac{\partial Z_2}{\partial B_2} = 1$

We want:

- B_2 is `(n_output)`

from:

- $\frac{\partial \mathcal{L}}{\partial Z_2} = \Delta_2$ is `(n_examples, n_output)`

If we average down the columns (axis=0) we'll get `(n_output)` to update B_2

Overall:

$$B_2 \leftarrow B_2 - \eta \cdot \frac{1}{N} \cdot \text{rowsum}(\Delta_2)$$

In [10]:
```
grad_B2 = Delta_2.mean(axis=0) # same as Delta_2.sum(axis=0) / n_examples
```

Fourth part, figuring out how to update W_1

$$W_1 \leftarrow W_1 - \eta \cdot \frac{\partial \mathcal{L}}{\partial W_1} \tag{5}$$

$$W_1 \leftarrow W_1 - \eta \cdot \frac{\partial \mathcal{L}}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial H_1} \cdot \frac{\partial H_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial W_1} \tag{6}$$

Recall $Z_1 = XW_1^\top + B_1;\ Z_2 = H_1 W_2^\top + B_2;\ H_1 = \sigma(Z_1)$ so roughly we have:

- $\frac{\partial \mathcal{L}}{\partial Z_2} = \Delta_2$ `(n_examples, n_output)`
- $\frac{\partial Z_2}{\partial H_1} = W_2$ `(n_output, n_hidden)`
- $\frac{\partial H_1}{\partial Z_1} = \sigma(Z_1) \cdot (1 - \sigma(Z_1))$ `(n_examples, n_hidden; applies element wise)`
- $\frac{\partial Z_1}{\partial W_1} = X$ `(n_examples, n_input)`

We want:

- W_1 is `(n_hidden, n_input)`

So:

- $\Delta_2 W_2$ is `(n_examples, n_hidden)`
- $\Delta_1 \stackrel{\text{def}}{=} (\Delta_2 W_2) \odot (\sigma(Z_1) \cdot (1 - \sigma(Z_1)))$ is `(n_examples, n_hidden)`
- $\Delta_1^\top$ is `(n_hidden, n_examples)`

Overall:

$$W_1 \leftarrow W_1 - \eta \cdot \frac{1}{N} \cdot \Delta_1^\top \cdot X$$

In [11]:
```
Delta_1 = (Delta_2 @  W_2) * sigmoid(Z_1) * (1 - sigmoid(Z_1))
grad_W1 = (Delta_1.T @ X) / n_examples
```

Final part thank goodness, figuring out how to update B_1

$$B_1 \leftarrow B_1 - \eta \cdot \frac{\partial \mathcal{L}}{\partial B_1} \tag{7}$$

$$B_1 \leftarrow B_1 - \eta \cdot \frac{\partial \mathcal{L}}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial H_1} \cdot \frac{\partial H_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial B_1} \tag{8}$$

We know that $\frac{\partial Z_1}{\partial B_1} = 1$ and we can steal Δ_1 from the W_1 update equations, so now we just need to figure out the shapes:

- Δ_1 is `(n_examples, n_hidden)`

We want:

- B_1 is `(n_hidden)`

And we just do the same as in the third part: if we average down the columns (axis=0) we'll get `(n_hidden)` to update B_1

Overall:

$$B_1 \leftarrow B_1 - \eta \cdot \frac{1}{N} \cdot \text{rowsum}(\Delta_1)$$

In [12]:
```
grad_B1 = Delta_1.mean(axis=0) # same as Delta_1.sum(axis=0) / n_examples
```

Appendix 5 Task 2 running_nn.ipynb

Jupyter notebook for testing/benchmark the NN

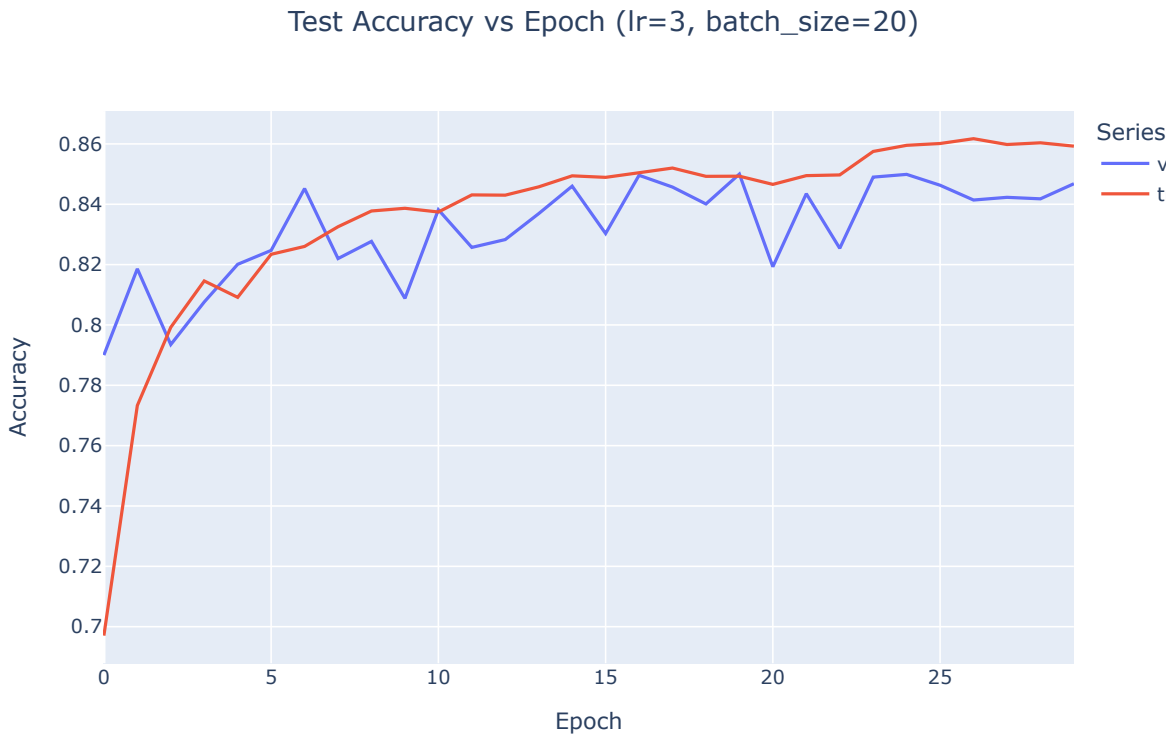
In [1]:
```
from nn import Model, ModelMetrics, init_model, train, eval, sep_labels, batch
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
import plotly
from tqdm import tqdm # for progress bar
import itertools
plotly.offline.init_notebook_mode()

n_input = 784
n_hidden = 30
n_output = 10
classes = 10

d_train = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_train.csv.gz", delimiter=',', skiprows=1), classes) # (60000, 785) first col is label
d_test = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_test.csv.gz", delimiter=',', skiprows=1), classes) # (10000, 785) first col is label
```

```
(60000, 10) (60000, 784)
(10000, 10) (10000, 784)
```

Part 1: epochs = 30, mini-batch size = 20, η = 3; graph of test accuracy vs epoch; print max accuracy

In [4]:
```
%%timeit -n1 -r1

model = init_model(n_input, n_hidden, n_output)
metrics = ModelMetrics()

lr = 3
n_epochs = 30
batch_size = 20

data_1 = {'epoch': np.arange(n_epochs), 'val': [], 'train': []}

for i in tqdm(range(n_epochs)):
    # mb mini batch
    for mb in batch(d_train, batch_size):
        train(mb, model, metrics, lr)
    data_1['train'].append(metrics.accuracy.value)
    metrics.reset()

    for mb in batch(d_test, batch_size):
        eval(mb, model, metrics)
    data_1['val'].append(metrics.accuracy.value)
    metrics.reset()

print(f"max acc={max(data_1['val']):.2f}")
fig = px.line(data_1, x='epoch', y=['val', 'train'],
                          labels={'value': 'Accuracy', 'epoch': 'Epoch', 'variable':'Series'},
                          title=f"Test Accuracy vs Epoch (lr={lr}, batch_size={batch_size})")
fig.update_layout(title={'x': 0.5, 'xanchor': 'center'})
fig.show()
```

```
100%|██████████| 30/30 [00:11<00:00,  2.66it/s]
max acc=0.85
```




```
13.4 s ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

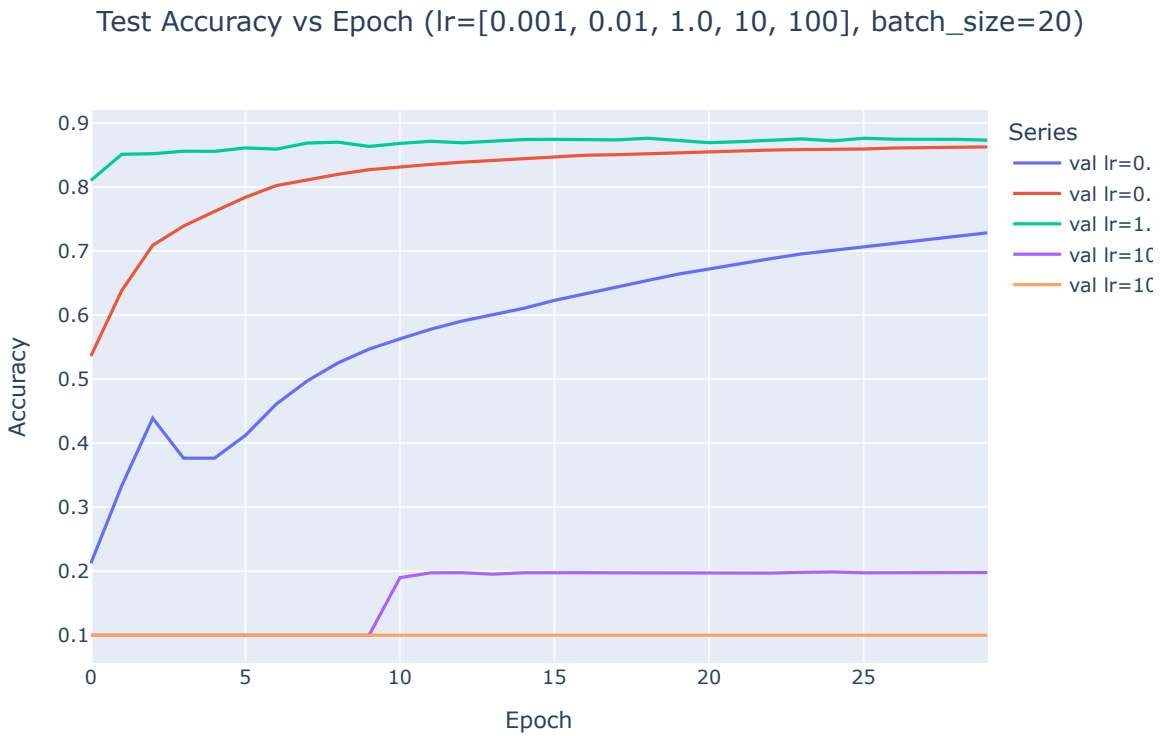
Part 2: epochs = 30, mini-batch size = 20, η = 0.001, 0.01, 1.0, 10, 100; graph of test accuracy vs epoch for each η; print max accuracy for each η

In [3]:
```
metrics = ModelMetrics()

learning_rates = [0.001, 0.01, 1.0, 10, 100]
n_epochs = 30
batch_size = 20

data_2 = {f'{k} lr={i}': [] for i in learning_rates for k in ["train", "val"]}
data_2['epoch'] = np.arange(n_epochs).tolist()

for lr, epoch in tqdm(list(itertools.product(learning_rates, range(n_epochs)))):
    if epoch == 0: model = init_model(n_input, n_hidden, n_output) # reset model
    # mb mini batch
    for mb in batch(d_train, batch_size):
        train(mb, model, metrics, lr)
    data_2[f'train lr={lr}'].append(metrics.accuracy.value)
    metrics.reset()

    for mb in batch(d_test, batch_size):
        eval(mb, model, metrics)
    data_2[f'val lr={lr}'].append(metrics.accuracy.value)
    metrics.reset()
```

```
 68%|▊▊▊▊▊▊▊   | 102/150 [00:43<00:22,  2.18it/s]/home/roman/learning-ml/2802ict-assignments/assignment_0
2/nn/nn.py:7: RuntimeWarning:

overflow encountered in exp

100%|▊▊▊▊▊▊▊▊▊▊| 150/150 [01:04<00:00,  2.32it/s]
```

In [4]:
```
print("\n".join(f'lr={i} max acc={max(data_2[f'val lr={i}']):.2f}' for i in learning_rates))

fig = px.line(data_2, x='epoch', y=[f'val lr={i}' for i in learning_rates],
                          labels={'value': 'Accuracy', 'epoch': 'Epoch', 'variable':'Series'},
                          title=f"Test Accuracy vs Epoch (lr={learning_rates},
batch_size={batch_size})")
fig.update_layout(title={'x': 0.5, 'xanchor': 'center'})
fig.show()
```

```
lr=0.001 max acc=0.73
lr=0.01 max acc=0.86
lr=1.0 max acc=0.88
lr=10 max acc=0.20
lr=100 max acc=0.10
```



Part 3: epochs = 30, mini-batch size = 1, 5, 20, 100, 300, η = 3; graph of test accuracy vs epoch for each mini-batch size; print max accuracy for each mini-batch

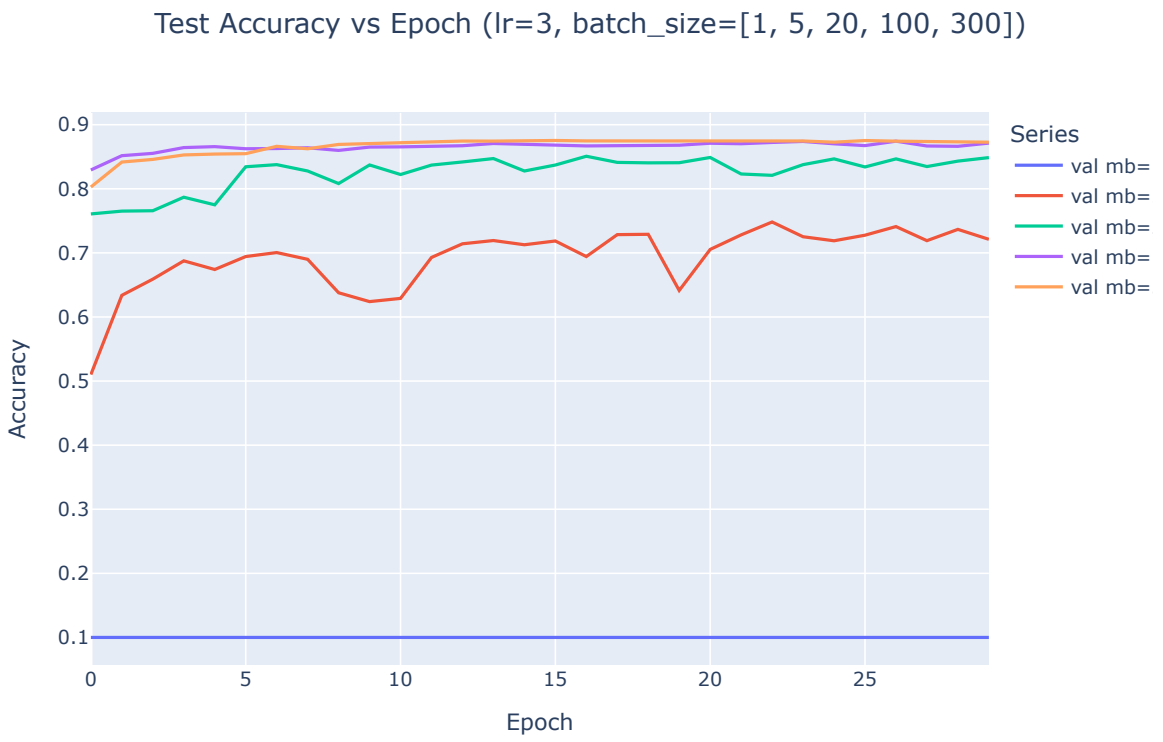
In [5]:
```
metrics = ModelMetrics()
model = init_model(n_input, n_hidden, n_output)

lr = 3
n_epochs = 30
batch_sizes = [1, 5, 20, 100, 300]

data_3 = {f'{k} mb={i}': [] for i in batch_sizes for k in ["train", "val"]}
data_3['epoch'] = np.arange(n_epochs).tolist()

for batch_size, epoch in tqdm(list(itertools.product(batch_sizes, range(n_epochs)))):
    if epoch == 0: model = init_model(n_input, n_hidden, n_output) # reset model
    # mb mini batch
    for mb in batch(d_train, batch_size):
        train(mb, model, metrics, lr)
    data_3[f'train mb={batch_size}'].append(metrics.accuracy.value)
    metrics.reset()

    for mb in batch(d_test, batch_size):
        eval(mb, model, metrics)
    data_3[f'val mb={batch_size}'].append(metrics.accuracy.value)
    metrics.reset()
```

```
  0%|          | 0/150 [00:00<?, ?it/s]/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/nn.p
y:7: RuntimeWarning:

overflow encountered in exp

100%|██████████| 150/150 [03:59<00:00,  1.60s/it]
```

In [6]:
```
print("\n".join(f'mb={i} max acc={max(data_3[f'val mb={i}']):.2f}' for i in batch_sizes))

fig = px.line(data_3, x='epoch', y=[f'val mb={i}' for i in batch_sizes],
                          labels={'value': 'Accuracy', 'epoch': 'Epoch', 'variable':'Series'},
```

```
                                title=f"Test Accuracy vs Epoch (lr={lr}, batch_size={batch_sizes})")
fig.update_layout(title={'x': 0.5, 'xanchor': 'center'})
fig.show()
```

```
mb=1 max acc=0.10
mb=5 max acc=0.75
mb=20 max acc=0.85
mb=100 max acc=0.87
mb=300 max acc=0.88
```



Appendix 6 Task 2 benchmarking-nn.ipynb

In [1]:
```
from nn import Model, ModelMetrics, init_model, train, eval, sep_labels, batch
import numpy as np

n_input = 784
n_hidden = 30
n_output = 10
classes = 10

d_train = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_train.csv.gz", delimiter=',', skiprows=1), classes) # (60000, 785) first col is label
d_test = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_test.csv.gz", delimiter=',', skiprows=1), classes) # (10000, 785) first col is label
```

```
(60000, 10) (60000, 784)
(10000, 10) (10000, 784)
```

In [2]:
```
model = init_model(n_input, n_hidden, n_output)
metrics = ModelMetrics()

lr = 3
n_epochs = 30
batch_size = 32
```

In [3]:
```
%%timeit -n5 -r2

def bench(model, metrics):
    for i in range(n_epochs):
        # mb mini batch
        for mb in batch(d_train, batch_size):
            train(mb, model, metrics, lr)

bench(model, metrics)
```

```
9.22 s ± 82.4 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)
```

Appendix 7 Task 2 benchmarking-nn-accelerated.ipynb

In [1]:
```
from nn_accelerated import init_model, ModelMetrics, sep_labels, to_device_batches, run_all_epochs,
run_epoch, train_step
import numpy as np
from jax import random
```

In [2]:
```
n_input = 784
n_hidden = 30
n_output = 10
classes = 10

d_train = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_train.csv.gz", delimiter=',', skiprows=1), classes) # (60000, 785) first col is label
d_test = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_test.csv.gz", delimiter=',', skiprows=1), classes) # (10000, 785) first col is label

batched_d_train = to_device_batches(d_train[0], d_train[1], 32)
batched_d_test = to_device_batches(d_test[0], d_test[1], 32)

key = random.key(0)
```

In [13]:
```
model = init_model(key, n_input, n_hidden, n_output)
metrics = ModelMetrics()

lr = 3
n_epochs = 30
batch_size = 20

## We run the model once to make sure JAX jits everything before we benchmark
run_all_epochs(model, metrics, batched_d_train[0], batched_d_train[1], n_epochs, lr)

print("Model ready!")
```

```
Model ready!
```

Benchmarking the epoch-vectorised approach:

In [15]:
```
%%timeit -n5 -r2

run_all_epochs(model, metrics, batched_d_train[0], batched_d_train[1], n_epochs, lr)
```

```
1.57 s ± 30.7 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)
```

Benchmarking the per-epoch vectorised approach:

In [14]:
```
%%timeit -n5 -r2

def bench(model, metrics):
    for i in range(n_epochs):
        model, metrics = run_epoch(model, metrics, batched_d_train[0], batched_d_train[1], lr)

bench(model, metrics)
```

```
1.54 s ± 26.3 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)
```

Benchmarking the per-minibatch vectorised approach:

In [18]:
```
%%timeit -n5 -r2

def bench(model, metrics):
    for i in range(n_epochs):
        for k in range(batched_d_train[0].shape[0]):
            model, metrics = train_step((batched_d_train[0][k], batched_d_train[1][k]), model, metrics,
lr)

bench(model, metrics)
```

```
1min 16s ± 627 ms per loop (mean ± std. dev. of 2 runs, 5 loops each)
```

Appendix 8 Task 2 hyperparameter-tuning.ipynb

In [27]:
```
from nn import Model, ModelMetrics, init_model, train, eval, sep_labels, batch
import numpy as np
from tqdm import tqdm

import plotly.graph_objects as go
import plotly.express as px
import plotly
plotly.offline.init_notebook_mode()
```

In [28]:
```
n_input = 784
# n_hidden = 30
n_output = 10
classes = 10

d_train = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_train.csv.gz", delimiter=',', skiprows=1), classes)
d_test = sep_labels(np.loadtxt("/home/roman/learning-ml/2802ict-assignments/assignment_02/nn/fashion-
mnist_test.csv.gz", delimiter=',', skiprows=1), classes)

lr = 0.1
n_epochs = 60
batch_size = 128
n_hidden = 128


model = init_model(n_input, n_hidden, n_output)
metrics = ModelMetrics()

accuracy = {'epoch': np.arange(n_epochs), 'val': [], 'train': []}
loss = {'epoch': np.arange(n_epochs), 'val': [], 'train': []}
```

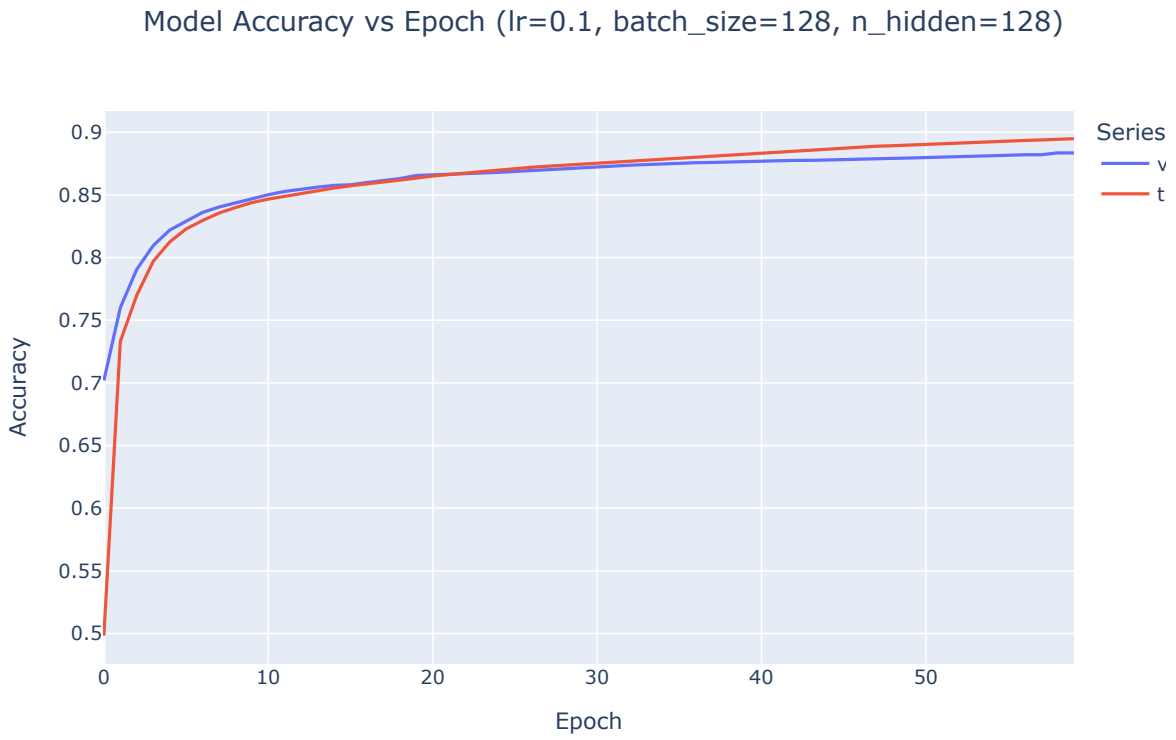
```
(60000, 10) (60000, 784)
(10000, 10) (10000, 784)
```

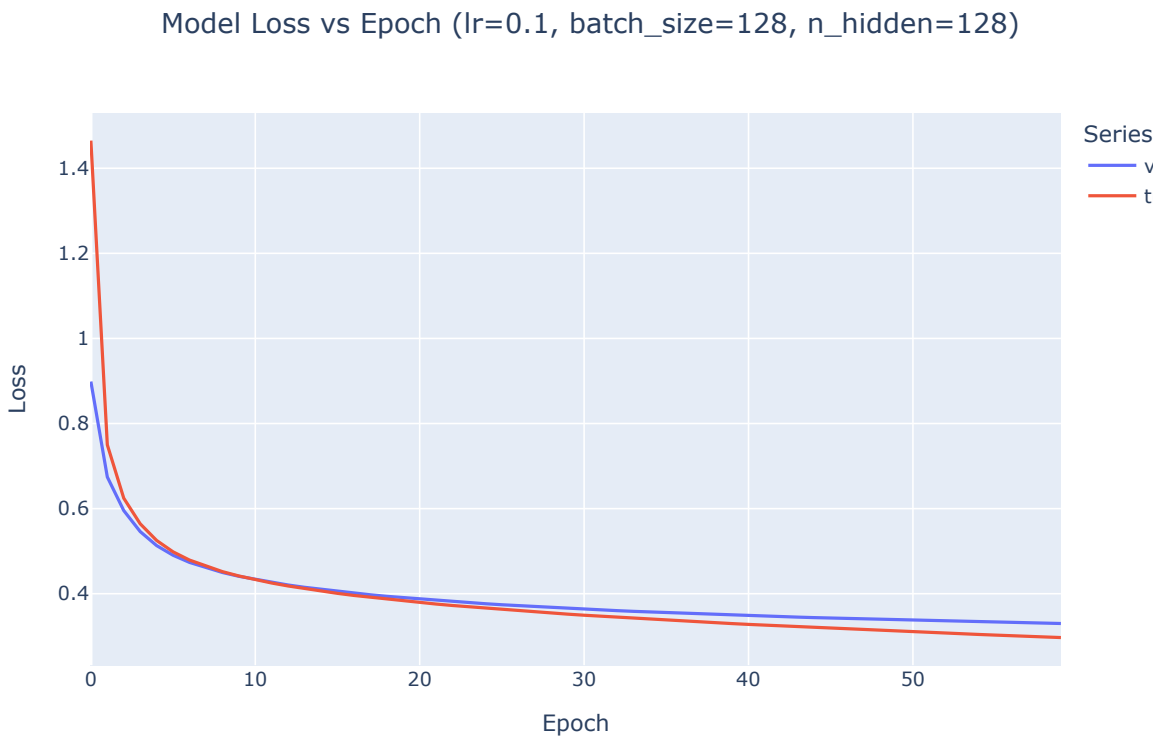
In [29]:
```
for i in tqdm(range(n_epochs)):
    # mb mini batch
    for mb in batch(d_train, batch_size):
        train(mb, model, metrics, lr)
    accuracy['train'].append(metrics.accuracy.value)
    loss['train'].append(metrics.loss.value)

    metrics.reset()

    for mb in batch(d_test, batch_size):
        eval(mb, model, metrics)
    accuracy['val'].append(metrics.accuracy.value)
    loss['val'].append(metrics.loss.value)
    metrics.reset()
```

```
100%|██████████| 60/60 [01:16<00:00,  1.28s/it]
```

In [33]:
```
print(f"max acc={max(accuracy['val']):.2f}")
fig = px.line(accuracy, x='epoch', y=['val', 'train'],
                          labels={'value': 'Accuracy', 'epoch': 'Epoch', 'variable':'Series'},
                          title=f"Model Accuracy vs Epoch (lr={lr}, batch_size={batch_size},
n_hidden={n_hidden})")
fig.update_layout(title={'x': 0.5, 'xanchor': 'center'})
fig.show()
```

```
max acc=0.88
```



In [35]:
```
print(f"min loss={min(loss['val']):.2f}")
fig = px.line(loss, x='epoch', y=['val', 'train'],
                          labels={'value': 'Loss', 'epoch': 'Epoch', 'variable':'Series'},
                          title=f"Model Loss vs Epoch (lr={lr}, batch_size={batch_size},
n_hidden={n_hidden})")
fig.update_layout(title={'x': 0.5, 'xanchor': 'center'})
fig.show()
```

```
min loss=0.33
```